

DigitalTwin for MyPalletizer 260 M5

Unity Version



Inhaltsverzeichnis

1. Abstract	2
2. Introduction and Goals	3
2.1. Functional Requirements	3
2.2. Quality Goals	3
2.3. Stakeholders	4
3. Architecture Constraints	5
3.1. Technische Randbedingungen	5
3.2. Organisatorische Randbedingungen	5
3.3. Konventionelle Randbedingungen	6
4. Context and Scope	6
4.1. Business Context	6
4.2. Technical Context	7
5. Solution Strategy	8
5.1. Architekturelle Leitprinzipien	8
6. Building Block View	9
6.1. Whitebox Overall System (Level 1)	9
6.2. Level 2: Subsysteme	10
6.3. Level 3: Detailbausteine	13
7. Runtime View	15
7.1. Szenario 1: Initialisierung und Verbindungsaufbau	16
7.2. Szenario 2: Asynchrone Gelenkbewegung (<code>send_angles</code>)	16
7.3. Szenario 3: Synchrone Gelenkbewegung (<code>sync_move_joints</code>)	17

7.4. Szenario 4: Dynamisches Objektpawnen (World.spawn)	18
7.5. Szenario 5: Endeffektor-Interaktion (Vakuumpumpe einschalten)	19
8. Deployment View	20
8.1. Infrastruktur- und Verteilungsdiagramm	20
8.2. Knoten und Softwareartefakte	21
8.3. Kommunikationskanäle	22
8.4. Bereitstellungsvarianten	22
8.5. Softwarepaketierung	22
9. Cross-cutting Concepts	22
9.1. Simulations- und Kinematikkonzept	22
9.2. Kommunikations- und Protokollkonzept	23
9.3. Threading- und Nebenläufigkeitskonzept in Unity	24
9.4. Zustands- und Telemetrikonzept	24
9.5. Endeffektor- und Greifkonzept	25
9.6. Welt- und Szenarienkonzept	25
9.7. Schutz- und Validierungskonzept	26
9.8. Test- und Qualitätssicherungskonzept	26
9.9. Usability und Developer Experience	26
10. Architecture Decisions	26
10.1. ADR-1: Einheitliche Python-API für Simulation und Hardware	26
10.2. ADR-2: Kapselung der Herstellerbibliothek pymycobot	27
10.3. ADR-3: Hybride Betriebsmodi (virtual, real, both)	27
10.4. ADR-4: Trennung der Kommunikationskanäle (UDP Ports 5005 und 5006)	28
10.5. ADR-5: Bereitstellung vordefinierter Szenarien über Presets	28
10.6. ADR-6: Einsatz von ArticulationBody für die Gelenkphysik in Unity	29
10.7. ADR-7: Pragmatische Nutzung herstellerseitiger CAD-Modelle mit Kompensation	29
10.8. ADR-8: Verzicht auf optische Kamerasimulation zugunsten der Kernkinematik	30
11. Quality Requirements	30
11.1. Quality Tree	30
11.2. Quality Scenarios	31
11.3. Architektonische Zielkonflikte (Trade-offs)	32
12. Risks and Technical Debts	33
12.1. Architektur-Risiken	33
12.2. Technologische und Implementierungsrisiken	33
12.3. Technische Schulden und Weiterentwicklung	34
13. Glossary	34

1. Abstract

Dieses Dokument beschreibt die Software- und Systemarchitektur des digitalen Zwillings für den Roboterarm **MyPalletizer 260 M5** nach dem arc42-Standard. Das System ermöglicht die

Entwicklung, den Test und die hybride Steuerung von Simulation (Unity) und physischer Roboterhardware über eine einheitliche, benutzerfreundliche Python-Schnittstelle. Es wurde im Rahmen des Moduls Applied Artificial Intelligence (APPLAI) an der Hochschule Luzern (HSLU) entwickelt.

2. Introduction and Goals

Das Projekt **DigitalTwin for MyPalletizer 260 M5** entstand im Rahmen des Moduls Applied Artificial Intelligence (APPLAI) an der Hochschule Luzern (HSLU).

Ziel ist die Bereitstellung eines digitalen Zwillings für den 4-Achs-Palettierroboter **MyPalletizer 260 M5**. Die 3D-Simulationsumgebung ermöglicht das gefahrlose Entwickeln, Testen und Optimieren von Roboterprogrammen und Manipulationsszenarien in einer virtuellen Umgebung, bevor der Code auf der physischen Hardware ausgeführt wird. Dadurch werden Kollisionen und mechanische Beschädigungen an den realen Servomotoren verhindert.

Das System löst den Engpass der beschränkten Hardwareverfügbarkeit im Lehrbetrieb, indem es Studierenden das ortsunabhängige Programmieren ermöglicht. Als quelloffenes Projekt (MIT-Lizenz) steht die Lösung zudem externen Forschenden und Entwicklern zur Verfügung.

2.1. Functional Requirements

Anforderung	Beschreibung
3D-Visualisierung	Darstellung der Robotergeometrie, Gelenkbewegungen und Umgebungsobjekte in einer echtzeitfähigen 3D-Simulation (Unity).
Einheitliche Programmierschnittstelle	Bereitstellung einer Python-API, die identische Steuerbefehle für Simulation und physische Hardware akzeptiert, sodass Steuerungsskripte ohne Codeanpassung übertragbar sind.
Werkzeugunterstützung	Visuelle und funktionale Nachbildung der Endeffektoren: Parallelgreifer (Gripper) und Vakuumpumpe (Suction Pump).
Dynamische Szenenverwaltung	Erzeugen, Manipulieren und Löschen geometrischer Objekte (Würfel, Kugeln, Zylinder) zur Laufzeit sowie Laden vordefinierter Übungsszenarien (Presets).
Flexible Betriebsmodi	Unterstützung von drei Betriebsarten: virtual (reine Simulation), real (reine Hardware) und both (synchroner Parallelbetrieb zum direkten Abgleich).
Schutz- und Validierungslogik	Automatisches Begrenzen von Gelenkwinkeln, Geschwindigkeiten und LED-Farbwerten auf die mechanischen Spezifikationen des Roboters (Software Clamping).

2.2. Quality Goals

Die Architektur priorisiert die folgenden Qualitätsziele gemäss ISO/IEC 25010:

Rang	Qualitätsmerkmal	Szenario	Architektonische Massnahme
1	Interoperabilität & Austauschbarkeit	Ein Steuerungsskript soll wahlweise die Simulation, den realen Roboter oder beide Systeme ansteuern können.	Kapselung der Zielsysteme hinter einer einheitlichen Fassade (Robot), welche die modusspezifische Verteilung intern vornimmt.
2	Zeitverhalten & Latenz	Kontinuierliche Gelenkbewegungen sollen flüssig und ohne sichtbare Verzögerung in der 3D-Szene visualisiert werden.	Verwendung von verbindungslosen UDP-Datagrammen mit absoluten Zustandswerten sowie Entkopplung des Netzwerkempfangs vom Rendering über eine interne Warteschlange.
3	Zuverlässigkeit & Fehlertoleranz	Ungültige Parameter oder temporärer Paketverlust dürfen weder das Programm abbrechen noch zu unkontrollierten Bewegungen führen.	Parameterprüfung beim API-Aufruf (Fail-Fast), Software-Clamping für Grenzwerte und idempotente Zustandspakete.
4	Wartbarkeit & Modularität	Neue Werkzeuge oder Simulationsszenarien sollen ergänzt werden können, ohne die Kernkinematik zu verändern.	Strikte Funktionstrennung zwischen Robotersteuerung (Robot) und Umgebungsverwaltung (World) über getrennte Netzwerkkanäle.
5	Portabilität	Das Python-Paket und die Unity-Simulation müssen unter Windows, Linux und macOS lauffähig sein.	Verzicht auf plattformspezifische Bibliotheken und Bereitstellung plattformunabhängiger Builds.

2.3. Stakeholders

Rolle	Person / Gruppe	Hauptinteresse und Erwartung
Studierende	Endnutzer an der HSLU	Intuitive Python-API, verständliche Beispiele, gefahrloses Testen ohne Hardwarerisiko.
Dozierende	Prof. Dr. Jana Koehler (HSLU)	Zuverlässiges Werkzeug für Vorlesung und Praktika, saubere arc42-Dokumentation, reproduzierbare Übungsaufgaben.

Rolle	Person / Gruppe	Hauptinteresse und Erwartung
Entwickler	Colin Felber (HSLU)	Modulare Architektur, hohe Wartbarkeit und erfolgreicher Modulabschluss.
Nachfolgeprojekte	Zukünftige Projektgruppen (z. B. AI-ROBOTICS)	Dokumentierte Architekturentscheidungen (ADRs), klare Schnittstellen und einfache Erweiterbarkeit.
Open-Source-Community	Externe Robotik-Entwickler	Standardisierte Installation via pip, offene MIT-Lizenz und transparente Protokolldokumentation.

3. Architecture Constraints

Die folgenden technischen, organisatorischen und konventionellen Vorgaben bilden den unveränderlichen Rahmen für die Architektur des digitalen Zwillings.

3.1. Technische Randbedingungen

Randbedingung	Beschreibung und Auswirkung
Hardware-Plattform	Das System ist an den physischen 4-Achs-Roboter MyPalletizer 260 M5 von Elephant Robotics gebunden. Dessen mechanische Achsgrenzen (J1: -160..160° , J2: 0..90° , J3: -92..60° , J4: -360..360°), Drehzahlen und Endeffektoren geben die Simulationsparameter vor.
Programmiersprache Python	Die Steuerungs- und Benutzerschnittstelle muss in Python (Version >= 3.12) entwickelt werden, um mit den an der Hochschule eingesetzten Entwicklungs- und Lehrmaterialien kompatibel zu sein.
Simulations-Engine Unity	Als 3D-Engine ist Unity (C#) festgelegt. Die Gelenkphysik muss auf der Unity-Komponente ArticulationBody basieren, um Mehrkörpersysteme stabil simulieren zu können.
Plattformunabhängigkeit	Sowohl das Python-Paket als auch die kompilierte Unity-Simulation müssen ohne systemspezifische Anpassungen auf Windows, Linux und macOS lauffähig sein.
Schnittstellen und Protokolle	Lokale Interprozesskommunikation erfolgt über UDP-Sockets. Die serielle Anbindung der Hardware erfordert einen USB-UART-Port mit 115200 Baud und die Bibliothek pymycobot .

3.2. Organisatorische Randbedingungen

Randbedingung	Beschreibung und Auswirkung
Hochschul- und Semesterrahmen	Das Projekt wurde als Einzelprojektarbeit im Modul APPLAI an der Hochschule Luzern innerhalb eines Semesters umgesetzt.

Randbedingung	Beschreibung und Auswirkung
Open-Source-Lizenz	Der gesamte Quellcode ist unter der MIT-Lizenz auf GitHub veröffentlicht. Sämtliche verwendeten Drittanbieter-Bibliotheken müssen mit dieser Lizenz kompatibel sein.

3.3. Konventionelle Randbedingungen

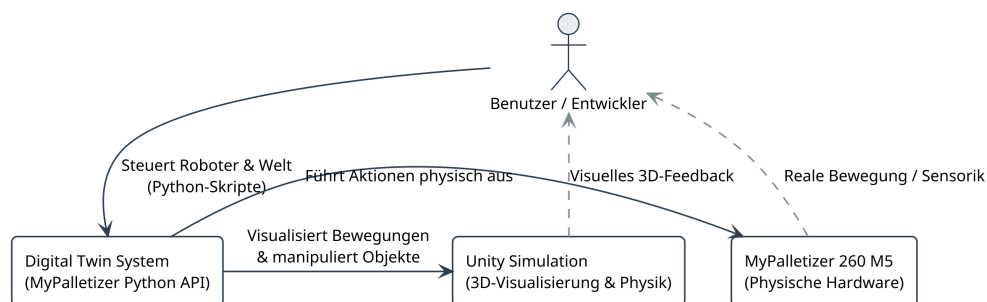
Randbedingung	Beschreibung und Auswirkung
Dokumentationsstandard arc42	Die Architekturbeschreibung folgt strukturell und inhaltlich dem arc42-Template . Als Dokumentationsformat wird AsciiDoc mit PlantUML verwendet.
Code- und Versionsrichtlinien	Der Python-Quellcode folgt dem PEP 8 -Styleguide mit Typannotationen (Type Hints); der Unity-Code folgt den Unity C# Coding Conventions. Die Versionskontrolle erfolgt mit Git und automatisierten GitHub Actions.

4. Context and Scope

Dieses Kapitel grenzt das System gegenüber externen Akteuren und angebundenen Nachbarsystemen ab. Es definiert die fachlichen Interaktionsbeziehungen (**Business Context**) sowie die technischen Kommunikationswege und Protokolle (**Technical Context**).

4.1. Business Context

Der fachliche Kontext beschreibt die Systemgrenzen aus der Anwenderperspektive. Der Benutzer steuert das Gesamtsystem ausschliesslich über die bereitgestellte Python-Bibliothek. Das System fungiert als Vermittler, validiert Parameter und leitet Aktionen je nach gewähltem Modus an die 3D-Simulation und/oder die physische Roboterhardware weiter.

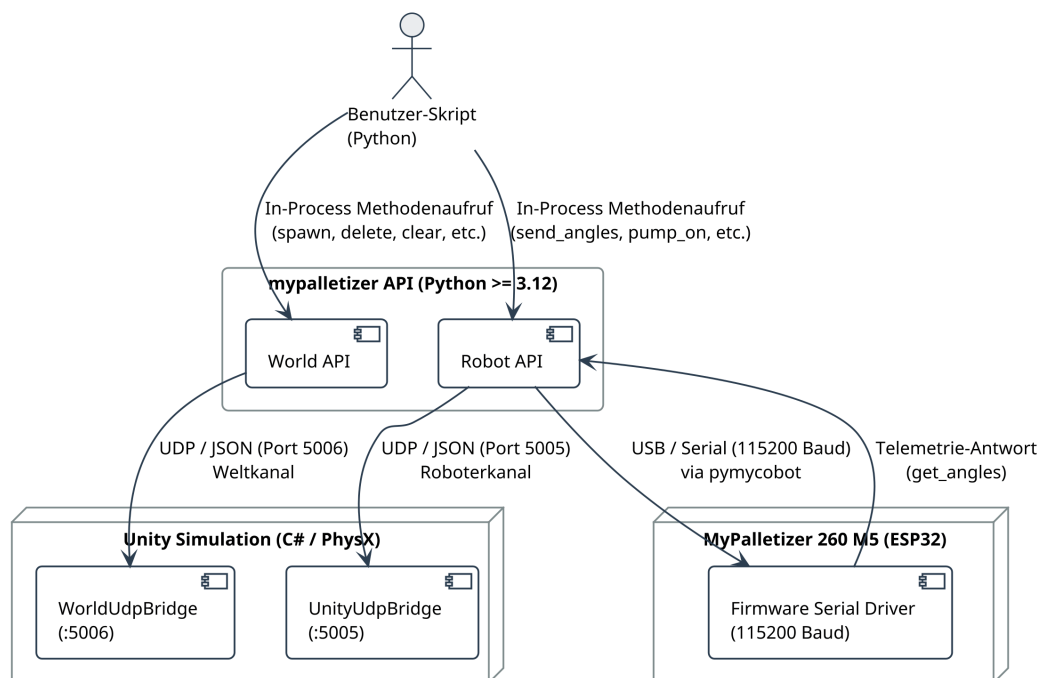


Akteur / Nachbarsystem	Fachliche Rolle	Fachliche Ein- und Ausgaben
Benutzer / Entwickler	Schreibt Steuerungsskripte in Python; plant Bewegungsabläufe und Pick-and-Place-Szenarien.	Eingabe: Steuerbefehle (Winkel, Geschwindigkeiten, Werkzeugaktionen, Szenenobjekte). Ausgabe: Visuelles Feedback in der Simulation und/oder physische Bewegung der Hardware.

Akteur / Nachbarsystem	Fachliche Rolle	Fachliche Ein- und Ausgaben
Digital Twin System (Python API)	Kapselt die interne Ablaufsteuerung, prüft Grenzwerte und steuert die Zielsysteme modusspezifisch an.	Eingabe: API-Methodenaufrufe des Benutzers. Ausgabe: JSON-Datagramme an Unity; serielle Steuerpakete an die Hardware.
Unity Simulation	3D-Echtzeit-Visualisierung von Kinematik, Werkzeugen und dynamischen Simulationskörpern.	Eingabe: UDP-Steuerkommandos. Ausgabe: Grafische Darstellung der Bewegungen und physikalischen Greifvorgänge.
MyPalletizer 260 M5	Physischer Roboterarm zur realen Ausführung von Bewegungsabläufen.	Eingabe: Serielle Befehlspakete. Ausgabe: Physische Gelenkbewegungen und Rückmeldung des aktuellen Hardwarezustands.

4.2. Technical Context

Der technische Kontext definiert die konkreten Schnittstellen, Medien und Übertragungsprotokolle zwischen dem Python-Steuerungssystem und den Nachbarsystemen.



Schnittstelle	Übertragungsmedium	Protokoll / Format	Technische Spezifikation
Benutzerskript → Python API	In-Process Funktionsaufruf	Python 3 API (Objektorientiert)	Direkte Methodenaufrufe auf den Klassen Robot und World mit nativer Parameterübergabe und Typisierung.

Schnittstelle	Übertragungsmedium	Protokoll / Format	Technische Spezifikation
Python API → Unity (Roboterkanal)	UDP Socket (Localhost)	UTF-8 JSON, Port 5005	Übertragung zeitkritischer Roboterkommandos (<code>move_joints</code> , <code>sync_move_joints</code> , <code>led</code> , <code>set_end_effector</code> , <code>set_gripper_state</code> , <code>pump</code>).
Python API → Unity (Weltkanal)	UDP Socket (Localhost)	UTF-8 JSON, Port 5006	Übertragung von Szenenverwaltungsbefehlen (<code>spawn</code> , <code>delete</code> , <code>clear</code>) inklusive 3D-Transformationsvektoren und Farbdaten.
Python API ↔ MyPalletizer Hardware	USB-UART (Serielle Schnittstelle)	8N1, 115200 Baud (via <code>pymycobot</code>)	Bidirektionale Kommunikation: Senden von Zielwinkeln und Relaiszuständen; Auslesen von Sensor- und Gelenk-Telemetriedaten.

5. Solution Strategy

Die Lösungsstrategie fasst die grundlegenden architektonischen Entwurfsentscheidungen zusammen, mit denen die funktionalen Anforderungen und Qualitätsziele realisiert werden.

5.1. Architekturelle Leitprinzipien

- **Schichtenarchitektur und Entkopplung (Separation of Concerns):** Die Anwendungslogik ist strikt in zwei unabhängige Schichten getrennt:
 - **Python Control Layer:** Beinhaltet Benutzerschnittstelle, Eingabevalidierung, Software-Clamping und die modusspezifische Verteilungslogik.
 - **Unity Simulation Layer:** Fungiert als reine Darstellungs- und Physik-Engine ohne eigene Ablaufsteuerungslogik. Diese Entkopplung ermöglicht automatisierte Unit-Tests in Python ohne laufende 3D-Engine.
- **Fassaden- und Adaptermuster für einheitliche Steuerung:** Die Klassen `Robot` und `World` stellen eine einheitliche Fassade (**Facade Pattern**) bereit. Benutzerskripte verwenden dieselben Methodenaufrufe für alle drei Betriebsmodi (`virtual`, `real`, `both`). Die herstellerspezifische Hardwarebibliothek `pymycobot` sowie die Netzwerk-Sockets sind vollständig im `MyPalletizerController` gekapselt (**Adapter Pattern**).
- **Verbindungsloses UDP mit absoluten Zustandswerten:** Um eine flüssige 3D-Darstellung ohne Latenzspitzen zu garantieren, wird verbindungsloses UDP eingesetzt. Jedes Bewegungs-Datagramm überträgt die vollständigen absoluten Winkel aller vier Achsen (`j1` bis `j4`). Dadurch ist ein einzelner Paketverlust unkritisch, da nachfolgende Pakete die Roboterpose unverzüglich korrigieren (Idempotenz).
- **Kanal- und Porttrennung zur Latenzoptimierung:** Um zu verhindern, dass umfangreiche Weltbefehle (z. B. das Erzeugen komplexer Szenarien) zeitkritische Roboterbewegungen blockieren, sind die Kommunikationswege auf zwei separate Ports aufgeteilt:
 - **Port 5005 (Roboterkanal):** Hochfrequente Gelenkbewegungen, LED-Farben und Werkzeugbefehle.

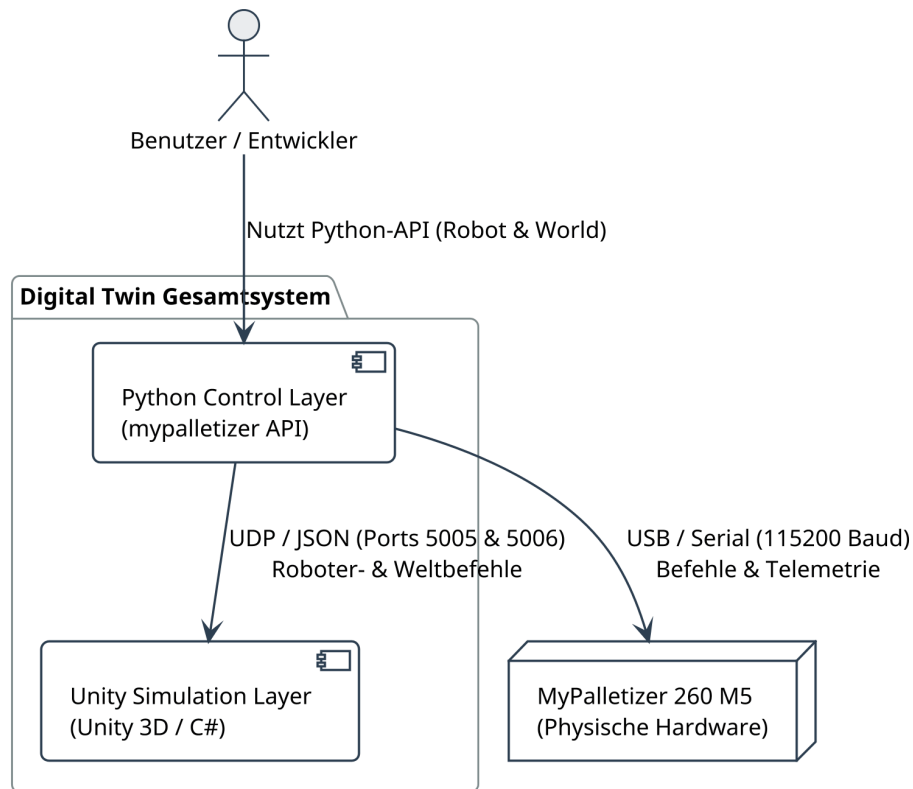
- **Port 5006 (Weltkanal):** Diskrete Ereignisse zur Objekt- und Szenenverwaltung (`spawn`, `delete`, `clear`).
- **Thread-sicheres Dispatching im Engine-Loop:** Da Unity Zugriffe auf die Szenenhierarchie ausschliesslich im Engine-Hauptthread (**Unity Main Thread**) erlaubt, werden empfangene UDP-Pakete in einem Hintergrund-Thread in threadsichere Warteschlangen eingereiht. Ein **CommandRunner** (Unity-Coroutine) holt die Kommandos synchron im Frame-Zyklus ab und führt sie aus.
- **Mehrstufiges Schutz- und Validierungskonzept:**
 - **Fail-Fast:** Fehlerhafte Konfigurationen (z. B. fehlender Port im Hardwarebetrieb) oder unvollständige 3D-Vektoren werden bei der Instanziierung sofort mit einer Exception abgefangen.
 - **Software Clamping:** Gelenkwinkel, Geschwindigkeiten und LED-Farbwerte werden vor der Übertragung automatisch auf die physikalischen Grenzen des MyPalletizer 260 M5 begrenzt.
- **Virtuelle Zustandshaltung (`_sim_angles`):** Im Simulationsbetrieb führt der Controller den Soll-Zustand im Array `_sim_angles` lokal mit. Die Abfragemethode `get_angles()` liefert dadurch auch ohne Netzwerk-Rückkanal sofort konsistente Winkelwerte zurück.

6. Building Block View

Die Bausteinsicht zeigt die statische Struktur des Gesamtsystems auf drei Detaillierungsebenen: Gesamtsystem (Level 1), Subsysteme (Level 2) und Detailkomponenten (Level 3).

6.1. Whitebox Overall System (Level 1)

Das System besteht aus dem **Python Control Layer** (Steuerung, Validierung, API) und dem **Unity Simulation Layer** (3D-Visualisierung, Kinematik, Physik). Der physische **MyPalletizer 260 M5** ist als externes Zielsystem angebunden.

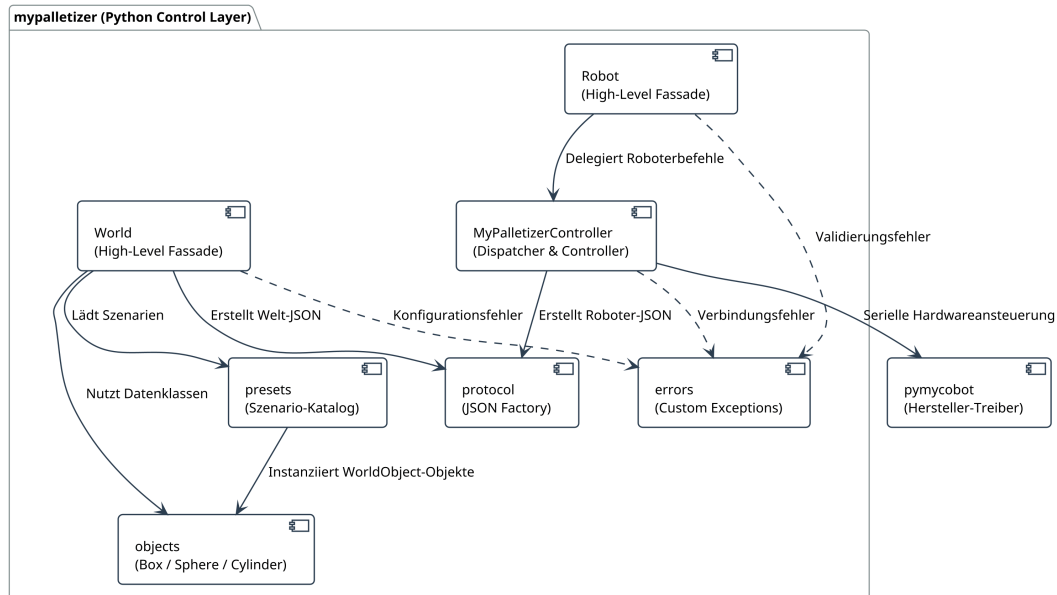


Baustein	Verantwortung und Schnittstellen
Python Control Layer	Öffentliche API für Benutzer; Validierung und Begrenzung von Steuerwerten; modusspezifische Weiterleitung an Simulation (UDP) und/oder Hardware (Serial).
Unity Simulation Layer	Empfang von UDP-Nachrichten; 3D-Visualisierung des Roboterarms und der Werkzeuge; physikalische Gelenkbewegung via <i>ArticulationBody</i> ; Verwaltung virtueller Objekte.
MyPalletizer 260 M5	Physische Hardware; Ausführung von Bewegungsbefehlen und Werkzeugaktionen über interne Servomotoren und Relais.
Benutzer / Entwickler	Erstellt Steuerungsskripte unter ausschliesslicher Verwendung der Klassen <i>Robot</i> und <i>World</i> .

6.2. Level 2: Subsysteme

6.2.1. Whitebox Python Control Layer

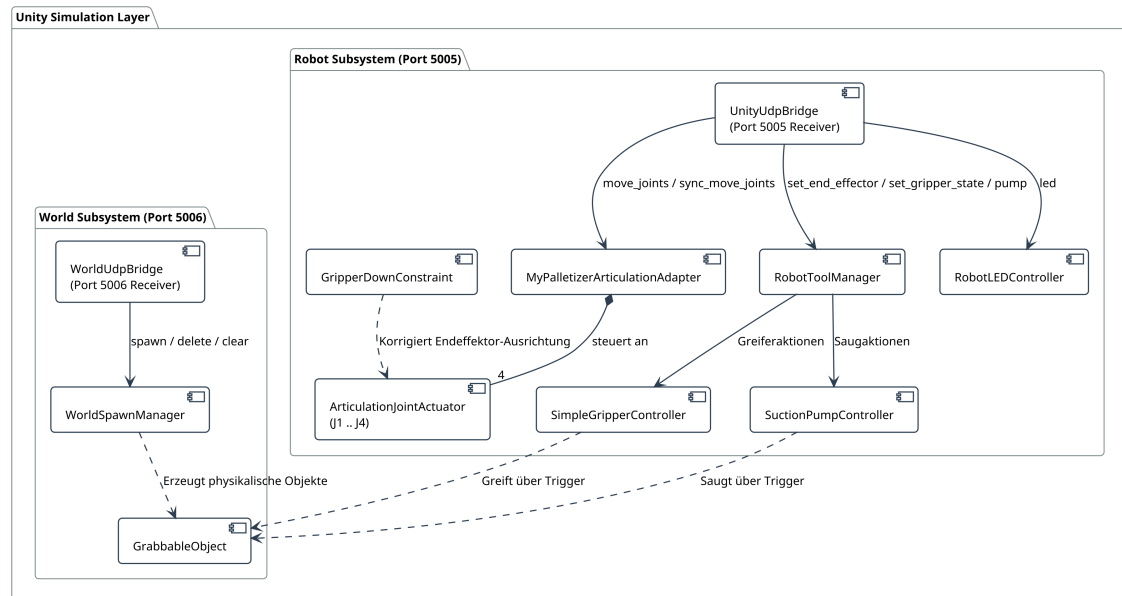
Der Python Control Layer gliedert sich in benutzernahe Fassadenklassen, den zentralen Controller sowie Hilfsmodule für Protokollerstellung, vordefinierte Presets, Datenklassen und Ausnahmen.



Baustein	Verantwortung und Aufgaben
Robot	High-Level-Fassade für die Robotersteuerung (<code>send_angles</code> , <code>sync_move_joints</code> , <code>set_color</code> , <code>set_end_effector</code> , <code>set_gripper_state</code> , <code>pump_on</code> , <code>pump_off</code>). Unterstützt das Python Context-Manager-Protokoll (<code>enter</code> , <code>exit</code>).
MyPalletizerController	Zentraler Dispatcher und Treiber. Führt Software-Clamping für Winkel, Geschwindigkeit und RGB durch; verwaltet den lokalen Zustandsspeicher <code>_sim_angles</code> ; hält den UDP-Socket für Port <code>5005</code> ; kapselt die <code>MyPalletizer260</code> -Hardwareinstanz.
World	High-Level-Fassade für die Umgebungsverwaltung (<code>spawn</code> , <code>delete</code> , <code>clear</code> , <code>load_preset</code>). Sendet über einen separaten UDP-Socket auf Port <code>5006</code> .
protocol	Zentrale Factory zur Konstruktion einheitlicher JSON-Nachrichten mit Versionsfeld (<code>"v": 1</code>) und Typkennung (<code>"type"</code>).
presets	Katalog vorgefertigter Szenarien (<code>empty</code> , <code>three_blocks</code> , <code>pick_and_place_basic</code> , <code>sorting_task</code>).
objects	Datenklassen für 3D-Körper: <code>WorldObject</code> (Basisklasse) sowie <code>Box</code> , <code>Sphere</code> und <code>Cylinder</code> .
errors	Benutzerdefinierte Ausnahmen (<code>InvalidConfigError</code> , <code>RobotConnectionError</code> , <code>WorldConnectionError</code>).
pymycobot	Externe Herstellerbibliothek zur seriellen Kommunikation mit dem MyPalletizer 260 M5.

6.2.2. Whitebox Unity Simulation Layer

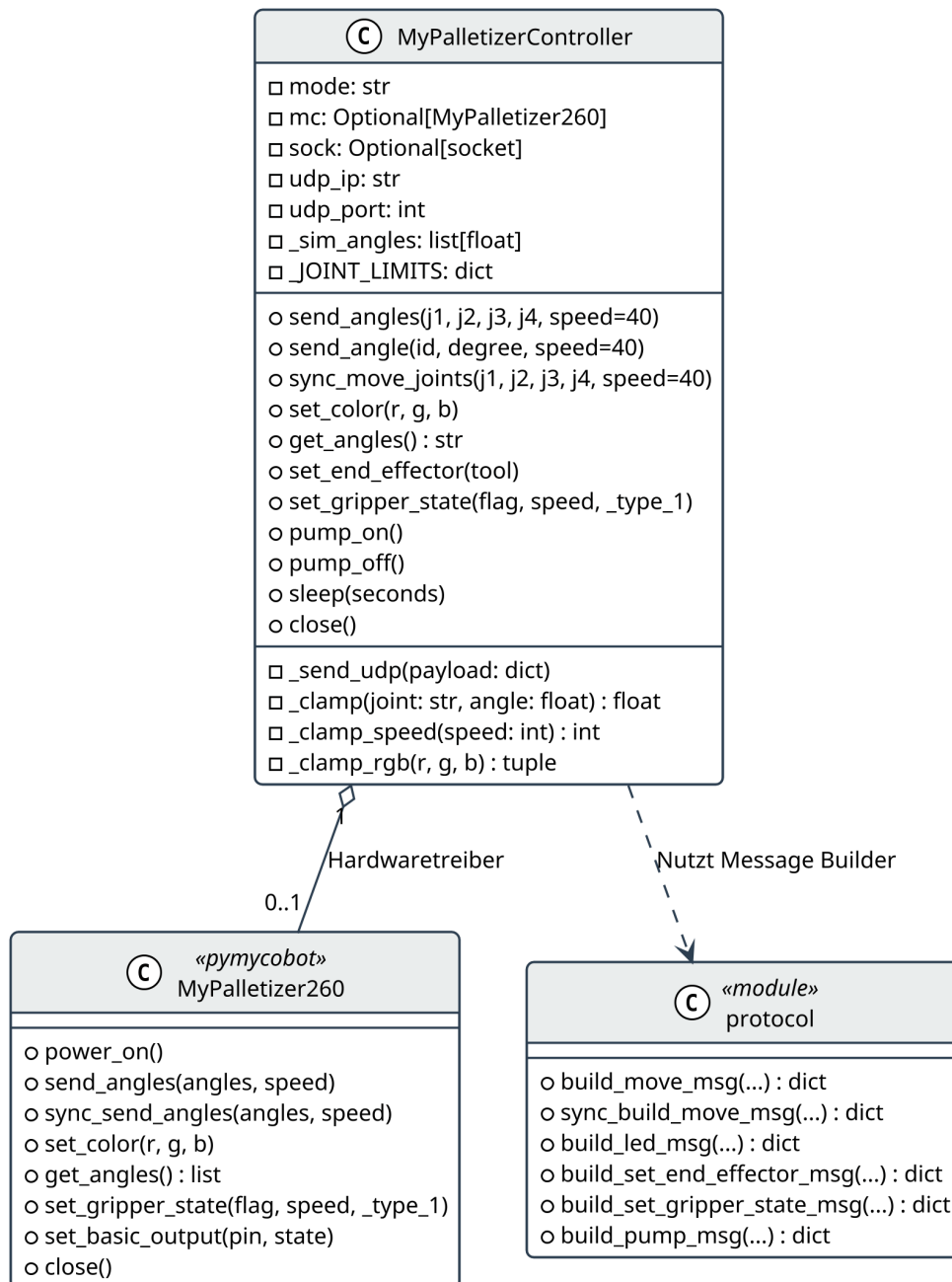
Die Simulationsarchitektur in Unity trennt strikt zwischen Roboter-Subsystem (Port `5005`) und Welt-Subsystem (Port `5006`).



Baustein	Verantwortung und Aufgaben
UnityUdpBridge	Empfängt UDP-Datagramme auf Port 5005 in einem Hintergrund-Thread und reiht sie threadsicher in eine Queue ein. Der CommandRunner leitet die Befehle im Hauptthread weiter.
MyPalletizerArticulationAdapter	Koordiniert die vier Gelenkaktuatoren j1 bis j4. Bietet MoveJointsAsync (asynchron) und MoveJointsSync (synchron via Coroutine).
ArticulationJointActuator	Steuert den ArticulationBody eines einzelnen Gelenks über dessen xDrive und führt die geschwindigkeitsabhängige Winkelinterpolation im FixedUpdate aus.
GripperDownConstraint	Hält den Werkzeugkopf im LateUpdate vertikal nach unten ausgerichtet (Kompensation der Palettiermechanik).
RobotToolManager	Schaltet die Sichtbarkeit der Endeffektor-Modelle (gripperTool / pumpTool) und leitet Aktionen an den aktiven Werkzeugcontroller weiter.
SimpleGripperController	Führt mechanische Greifvorgänge aus; erfasst GrabbableObject-Objekte via Physics.OverlapSphere und bindet sie kinematisch an den Greifer.
SuctionPumpController	Steuert die Vakuumpumpe; koppelt das nächstgelegene GrabbableObject bei aktivem Unterdruck an den Saugkopf.
RobotLEDController	Steuert die RGB- und Emissionsdarstellung der Basis-LEDs des simulierten M5-Moduls.
WorldUdpBridge	Empfängt UDP-Datagramme auf Port 5006 und übergibt sie im Update-Frame an den WorldSpawnManager.
WorldSpawnManager	Instanziert Prefabs (boxPrefab, spherePrefab, cylinderPrefab), konfiguriert Rigidbody und GrabbableObject, weist Farben zu und verwaltet aktive Objekte (_spawnedObjects).
GrabbableObject	Kennzeichnungskomponente für interaktionsfähige Objekte; verwaltet hierarchische Fixierung (AttachTo) und Freigabe (Detach).

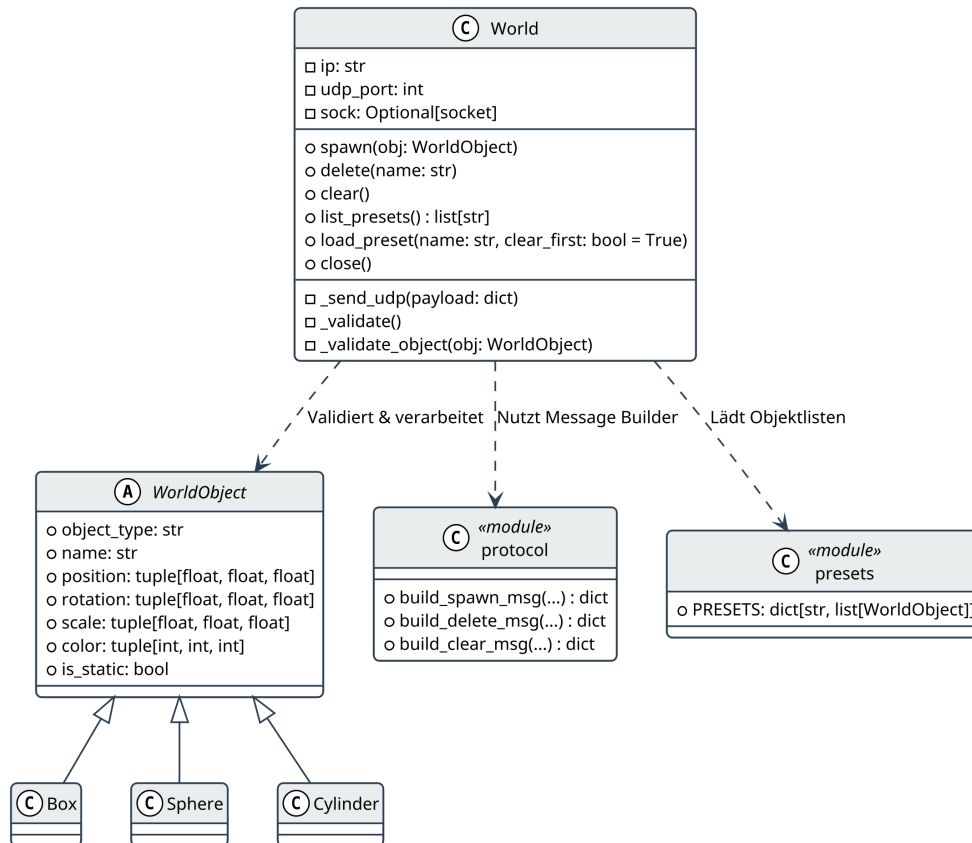
6.3. Level 3: Detailbausteine

6.3.1. MyPalletizerController (Klassendiagramm)



- **Gelenkwinkelbegrenzung (`_clamp`):** Schützt die Hardware durch hartcodierte Grenzwerte (`j1: -160..160°`, `j2: 0..90°`, `j3: -92..60°`, `j4: -360..360°`).
- **Telemetrie-Simulation (`get_angles`):** Gibt im Modus `virtual` die Werte aus `_sim_angles` und im Modus `real/both` die von der Hardware gelesenen Gelenkdaten zurück.

6.3.2. World und WorldObject (Klassendiagramm)

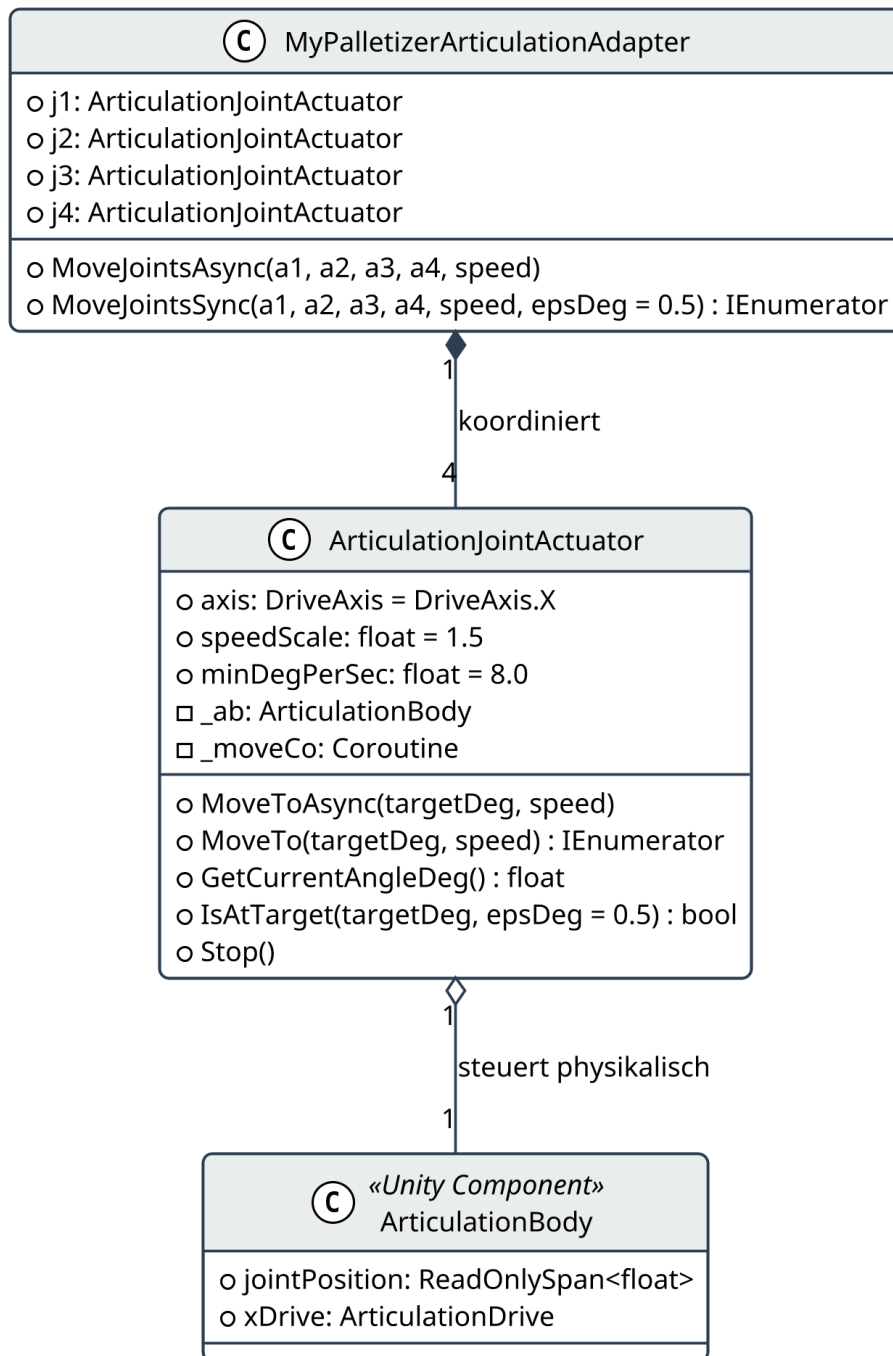


- **Validierung (`_validate_object`):** Prüft vor dem Senden, dass Objektnamen nicht leer sind, 3D-Vektoren 3 Elemente enthalten und Skalierungswerte strikt positiv (> 0) sind.

6.3.3. protocol (Nachrichtenformate)

Nachrichtentyp (type)	Port	JSON-Struktur
"move_joints"	5005	<code>{"v":1,"type":"move_joints","j1":0.0,"j2":20.0,"j3":-30.0,"j4":0.0,"speed":40}</code>
"sync_move_joints"	5005	<code>{"v":1,"type":"sync_move_joints","j1":10.0,"j2":45.0,"j3":0.0,"j4":90.0,"speed":50}</code>
"led"	5005	<code>{"v":1,"type":"led","r":0,"g":255,"b":0}</code>
"set_end_effector"	5005	<code>{"v":1,"type":"set_end_effector","tool":"gripper"}</code>
"set_gripper_state"	5005	<code>{"v":1,"type":"set_gripper_state","flag":1,"speed":50,"type_1":1}</code>
"pump"	5005	<code>{"v":1,"type":"pump","enabled":true}</code>
"spawn"	5006	<code>{"v":1,"type":"spawn","object_type":"box","name":"b1","position":{"x":0.1,"y":0.2,"z":0.3},"rotation":{"x":0,"y":0,"z":0},"scale":{"x":0.05,"y":0.05,"z":0.05},"color":{"r":255,"g":0,"b":0},"is_static":false}</code>
"delete"	5006	<code>{"v":1,"type":"delete","name":"b1"}</code>
"clear"	5006	<code>{"v":1,"type":"clear"}</code>

6.3.4. Gelenkaktuierung in Unity



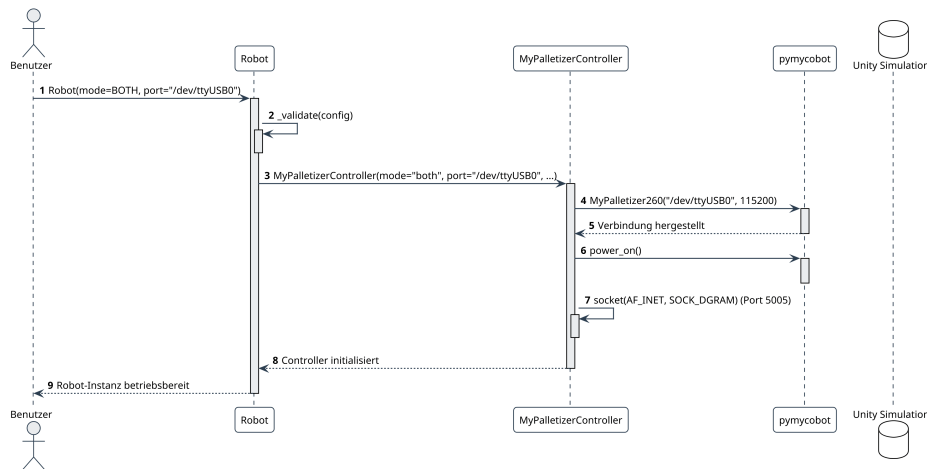
- **Winkelberechnung:** Der `ArticulationJointActuator` berechnet die Geschwindigkeit via $\text{degPerSec} = \max(\text{minDegPerSec}, \text{speed} * \text{speedScale})$ und führt die Rotation über `xDrive.target` im `FixedUpdate`-Zyklus nach.

7. Runtime View

Die Laufzeitsicht veranschaulicht das dynamische Zusammenspiel der Bausteine anhand der fünf wichtigsten Ablauf- und Interaktionsszenarien.

7.1. Szenario 1: Initialisierung und Verbindungsaufbau

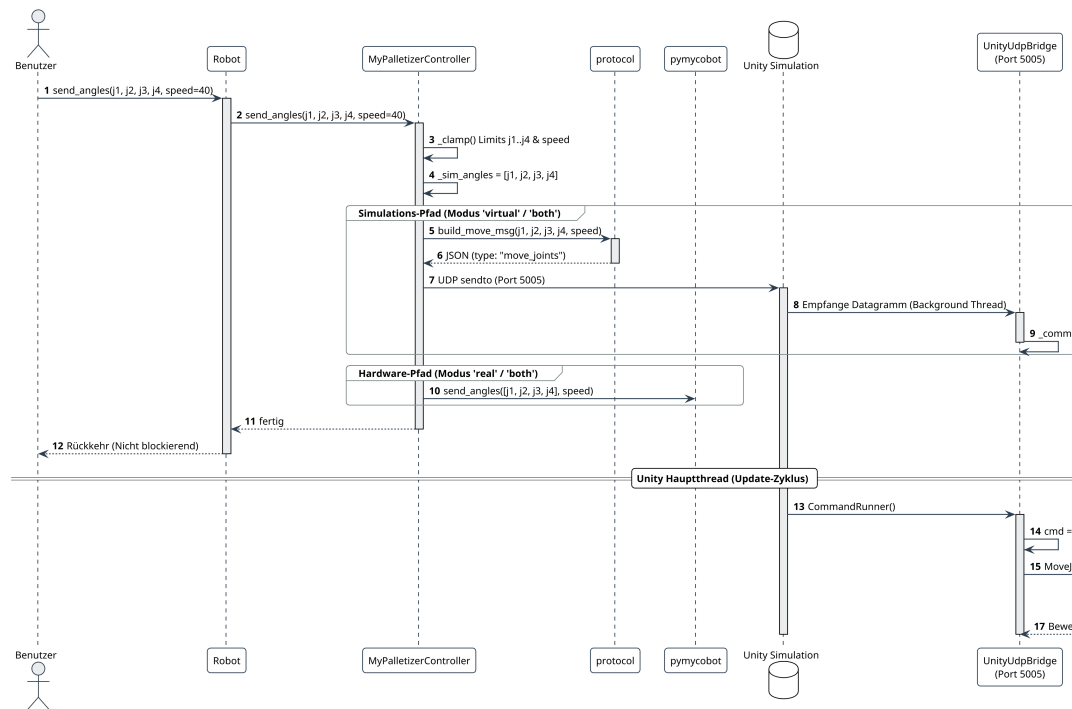
Dieses Szenario zeigt den Ablauf beim Starten eines Python-Skripts mit Initialisierung der Roboter- und Weltinstanzen (exemplarisch im Modus **both**).



- Instanziierung Robot:** Der Benutzer instanziert `Robot(mode=RobotMode.BOTH, port="/dev/ttyUSB0")`.
- Validierung:** `_validate()` stellt sicher, dass für die Modi **real** und **both** ein serieller Port definiert ist.
- Hardware-Initialisierung:** `MyPalletizerController` öffnet die serielle Verbindung via `pymycobot` (115200 Baud) und bestromt die Servomotoren mit `power_on()`.
- Netzwerk-Socket:** Der Controller öffnet einen UDP-Socket für die Robotersteuerung auf Port **5005**.
- Welt-Initialisierung:** Die Instanziierung von `World` öffnet einen separaten UDP-Socket auf Port **5006** für Szenenbefehle.

7.2. Szenario 2: Asynchrone Gelenkbewegung (`send_angles`)

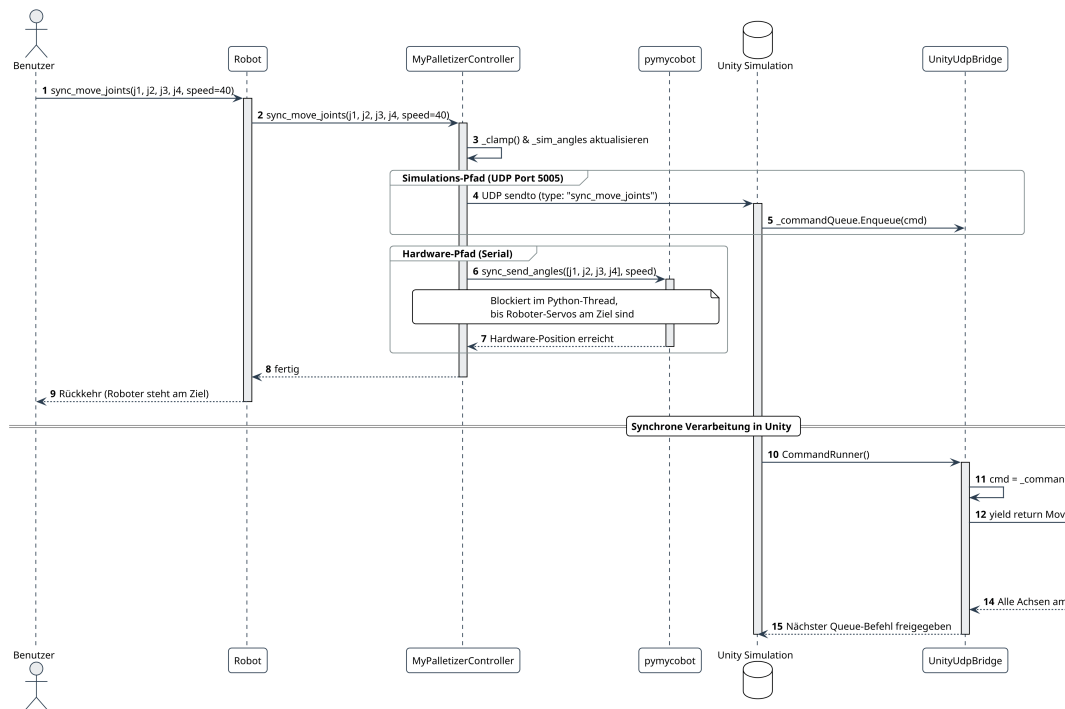
In diesem Szenario sendet der Benutzer Gelenkwinkel im "Fire-and-Forget"-Modus. Der Aufruf kehrt sofort zum Aufrufer zurück.



- API-Aufruf & Clamping:** Der Benutzer ruft `send_angles()` auf. Der Controller begrenzt Winkel und Geschwindigkeit und aktualisiert `_sim_angles`.
- JSON-Erstellung & UDP-Versand:** `protocol.build_move_msg()` erzeugt ein Datagramm (`type: "move_joints"`), das an Port 5005 gesendet wird.
- Hardwareübertragung:** Parallel dazu wird `send_angles()` an `pymycobot` übergeben.
- Abarbeitung in Unity:** `UnityUdpBridge` empfängt das Datagramm im Hintergrund-Thread und reiht es in `_commandQueue` ein. Der `CommandRunner` holt den Befehl im Hauptthread ab und übergibt die Zielwinkel an `MoveJointsAsync`, welche die Aktuatoren kontinuierlich nachführt.

7.3. Szenario 3: Synchrone Gelenkbewegung (`sync_move_joints`)

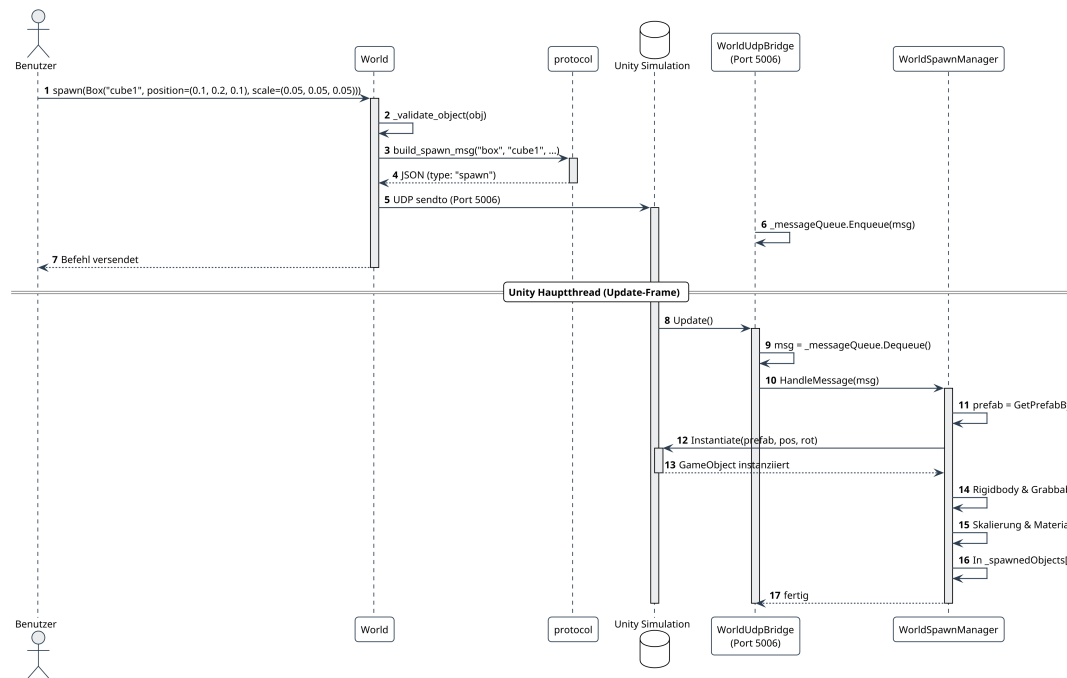
Dieses Szenario beschreibt eine blockierende Bewegung, bei welcher der Ablauf erst fortgesetzt wird, wenn alle Gelenke ihre Endposition eingenommen haben.



1. **API-Aufruf:** Der Benutzer ruft `sync_move_joints()` auf.
2. **Hardware-Synchronisation:** Der Controller ruft `sync_send_angles()` auf `pymycobot` auf. Diese Methode blockiert den Python-Thread, bis die physischen Servos die Zielposition erreicht haben.
3. **Simulations-Synchronisation:** Die `UnityUdpBridge` empfängt den Befehl (`type: "sync_move_joints"`). Der `CommandRunner` startet `yield return MoveJointsSync(...)`. Die Coroutine wartet in jedem Physik-Frame (`FixedUpdate`), bis alle vier `ArticulationBody`-Gelenke innerhalb der Toleranz (`epsDeg = 0.5°`) liegen. Erst danach wird der nächste Befehl aus der Warteschlange verarbeitet.

7.4. Szenario 4: Dynamisches Objektpawnen (`World.spawn`)

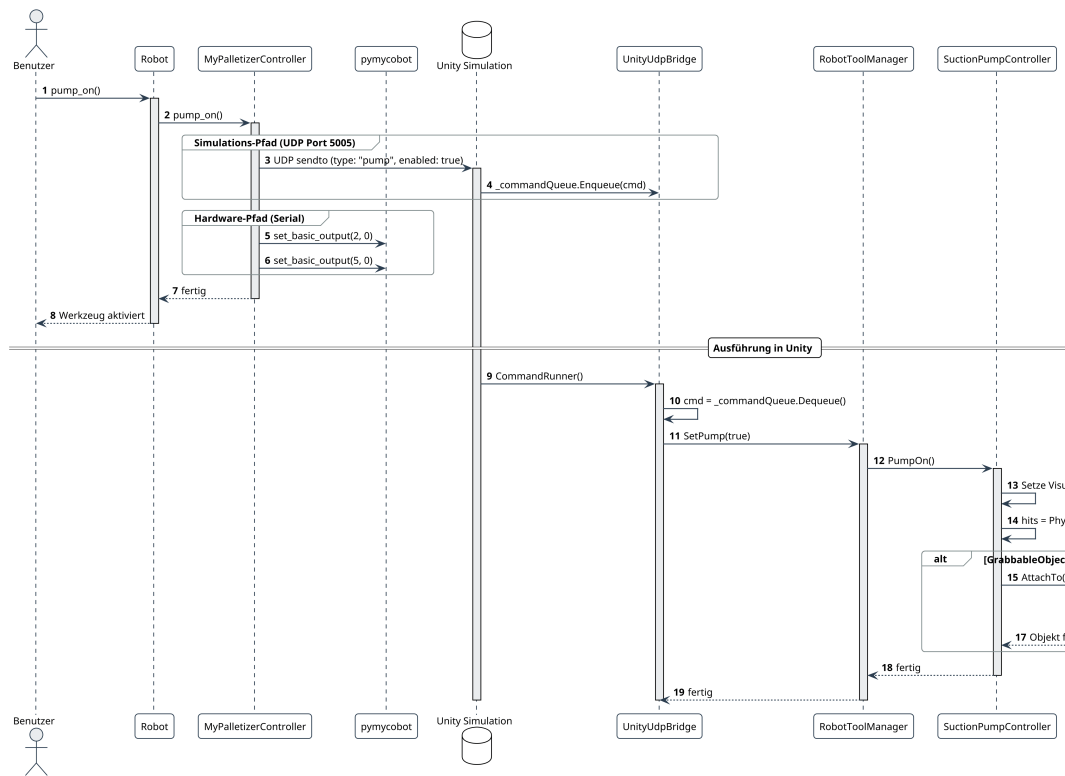
Dieses Szenario zeigt das Erzeugen und Konfigurieren eines geometrischen 3D-Körpers in der Simulationsumgebung.



- Objektdefinition & Prüfung:** Der Benutzer ruft `world.spawn(Box("cube1", ...))` auf. `_validate_object()` prüft Objektamen, Vektordimensionen und positive Skalierungswerte.
- Netzwerkübertragung:** `protocol.build_spawn_msg()` erstellt das JSON-Paket und sendet es an den Weltport 5006.
- Instanziierung in Unity:** `WorldUdpBridge` empfängt das Paket. Im nächsten `Update`-Frame instanziiert der `WorldSpawnManager` das passende Prefab, konfiguriert `Rigidbody` und `GrabbableObject`, setzt Skalierung sowie Materialfarbe und registriert das Objekt in `_spawnedObjects`.

7.5. Szenario 5: Endeffektor-Interaktion (Vakuumpumpe einschalten)

Dieses Szenario zeigt die Aktivierung der Vakuumpumpe und das physikalische Ansaugen eines manipulierbaren Objekts.



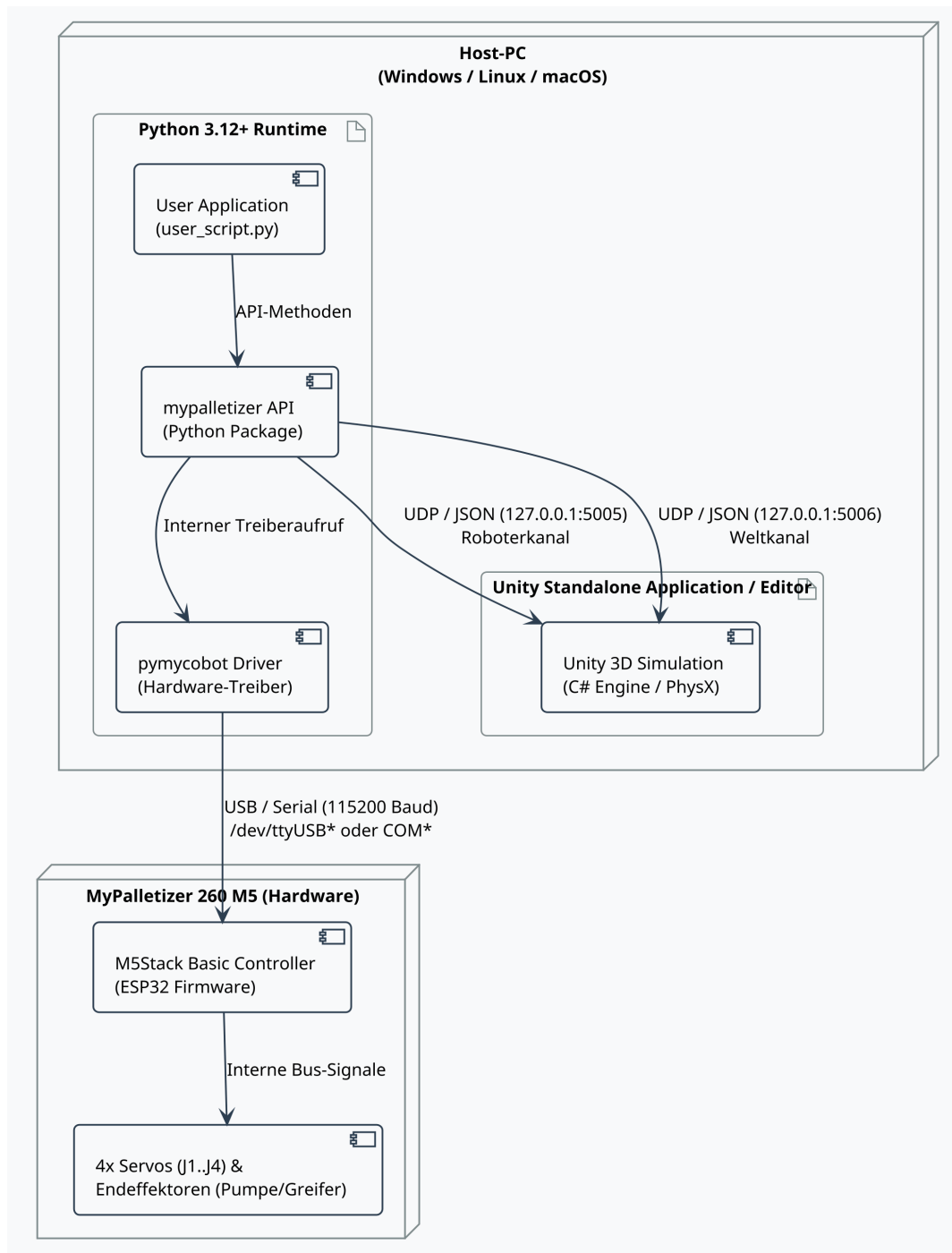
1. **API-Aufruf:** Der Benutzer ruft `robot.pump_on()` auf.
2. **Hardware-Aktivierung:** Der Controller schaltet die GPIO-Pins 2 und 5 über `pymycobot.set_basic_output()` auf Low (0), wodurch die physischen Pumpenrelais anziehen.
3. **Simulations-Aktivierung:** Ein UDP-Paket (`type: "pump", enabled: true`) wird an Port 5005 gesendet.
4. **Greiflogik in Unity:** Der `SuctionPumpController` aktiviert die optische Rückmeldung und sucht mittels `Physics.OverlapSphere` nach Objekten mit der Komponente `GrabbableObject`. Wird ein Objekt gefunden, wird es über `obj.AttachTo(attachPoint)` kinematisch an den Saugkopf gekoppelt.

8. Deployment View

Die Verteilungssicht beschreibt die physischen Hardwareknoten, die Software-Laufzeitumgebungen und die Netzwerkverbindungen des Gesamtsystems.

8.1. Infrastruktur- und Verteilungsdiagramm

Das System wird auf einem lokalen Arbeitsplatzrechner (**Host-PC**) ausgeführt. Optional ist der physische Roboter **MyPalletizer 260 M5** über eine serielle USB-Verbindung angeschlossen.



8.2. Knoten und Softwareartefakte

Knoten / Laufzeit	Typ	Enthaltene Komponenten und Voraussetzungen
Host-PC	Physischer Rechner	Standard-Entwicklungsrechner unter Windows (10/11), Linux (Ubuntu 22.04+) oder macOS (12+).
Python Runtime	Software-Umgebung (≥ 3.12)	Führt das Benutzerskript, das installierte Paket mypalletizer-api sowie die Abhängigkeiten numpy und pymycobot aus.
Unity Runtime	Standalone-Build / Unity Editor	Führt die kompilierte Simulationsanwendung aus; nutzt OpenGL/DirectX/Metal für das Rendering und PhysX für die Gelenkberechnung.

Knoten / Laufzeit	Typ	Enthaltene Komponenten und Voraussetzungen
MyPalletizer 260 M5	Embedded Hardware (ESP32)	M5Stack-Controller mit integrierter Echtzeit-Servosteuerung und Relaisausgängen für Endeffektoren.

8.3. Kommunikationskanäle

Kanal	Verbindung	Format / Baudrate	Zweck
Roboterkanal	UDP (127.0.0.1:5005)	UTF-8 JSON	Zeitkritische Übertragung von Gelenkwinkeln, LED-Farben und Werkzeugzuständen.
Weltkanal	UDP (127.0.0.1:5006)	UTF-8 JSON	Übertragung von Szenenkommandos (<code>spawn</code> , <code>delete</code> , <code>clear</code>).
Hardware-Kanal	USB-UART (Seriell)	8N1, 115200 Baud	Bidirektionale Ansteuerung der physischen Servomotoren und Relais über <code>pymycobot</code> .

8.4. Bereitstellungsvarianten

- **Reine Simulation (virtual):** Python-Skript und Unity-Anwendung laufen auf demselben Rechner; kein Hardwareanschluss erforderlich. Ideal für Fernunterricht und algorithmische Tests.
- **Hybrider Parallelbetrieb (both):** Python-Skript steuert zeitgleich die Unity-Simulation und den über USB verbundenen Roboter an. Dient dem direkten visuellen Soll-Ist-Vergleich.
- **Reiner Hardwarebetrieb (real):** Python-Skript steuert ausschliesslich den physischen Roboter an; Unity muss nicht gestartet werden.

8.5. Softwarepaketierung

- **Python-Bibliothek:** Paketierte als Standard-Wheel via `pyproject.toml` (`pip install .`).
- **Unity-Simulation:** Bereitstellung als eigenständige Standalone-Binaries für Windows (`.exe`), Linux (`.x86_64`) und macOS (`.app`) über automatisierte GitHub Actions.

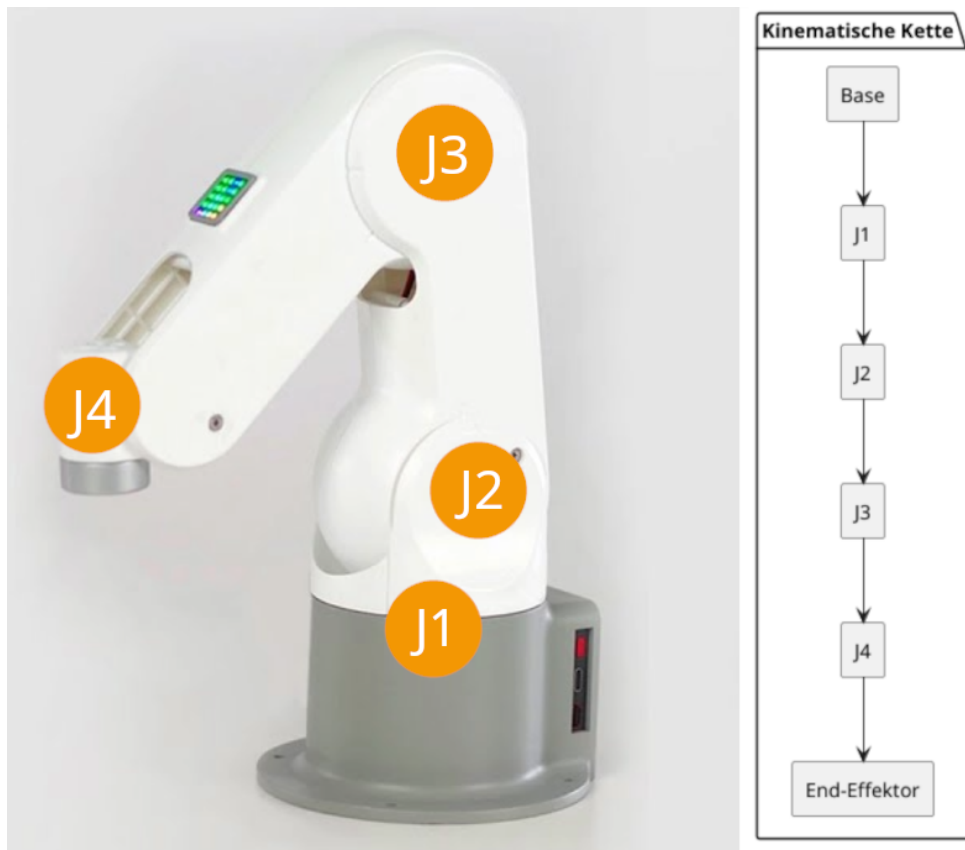
9. Cross-cutting Concepts

In diesem Kapitel werden die übergreifenden Entwurfsmuster, Architekturkonzepte und Implementierungsprinzipien beschrieben, die mehrere Bausteine des Systems prägen.

9.1. Simulations- und Kinematikkonzept

Die Kinematik des MyPalletizer 260 M5 wird in Unity über `ArticulationBody`-Komponenten abgebildet. Diese basieren auf dem Featherstone-Algorithmus der PhysX-Engine und bieten eine

exakte Berechnung serieller Mehrkörpersysteme ohne Gelenkdehnungen (**Joint Separation**).



- **Gelenkhierarchie:** Die vier Drehachsen (J1 bis J4) sind baumförmig angeordnet. Eine Drehung der Basis (J1) oder der Schulter (J2) bewegt alle nachfolgenden Glieder und Werkzeuge physikalisch korrekt mit.
- **Interpolation:** `ArticulationJointActuator` berechnet die erforderliche Schrittweite im `FixedUpdate`-Zyklus über `degPerSec = max(minDegPerSec, speed * speedScale)` und führt das Gelenk über `xDrive.target` flüssig nach.
- **Kompensation der Palettiermechanik:** Reale Palettierroboter besitzen ein mechanisches Parallelogrammgestänge, welches den Werkzeugkopf stets vertikal hält. In Unity wird dies im `LateUpdate`-Schritt über die Komponente `GripperDownConstraint` softwareseitig kompensiert.

9.2. Kommunikations- und Protokollkonzept

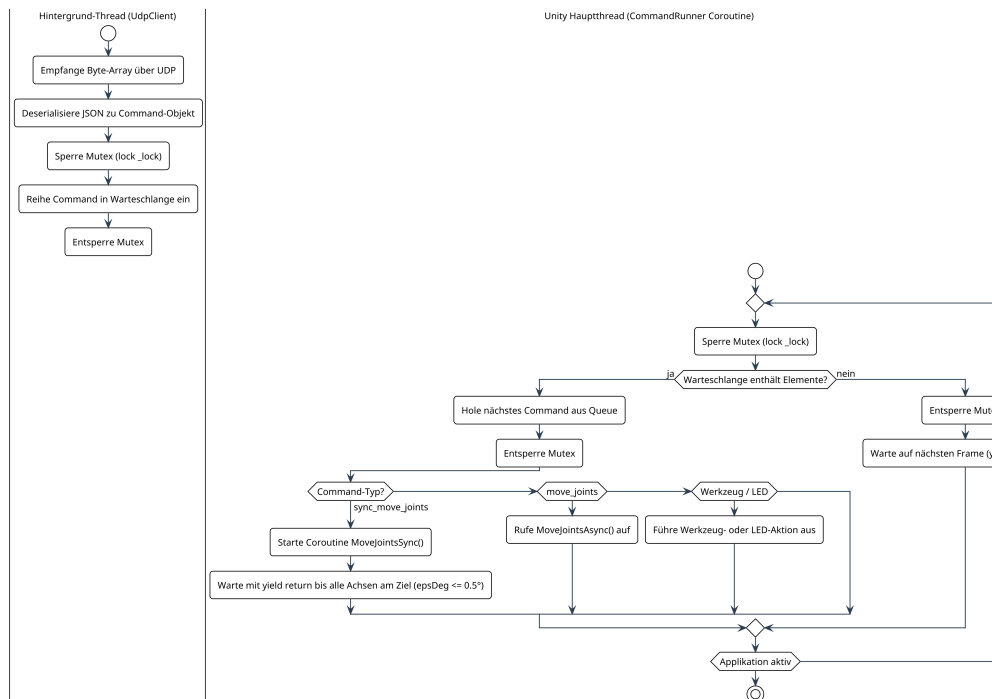
Die Netzwerkkommunikation basiert auf verbindungslosen UDP-Datagrammen mit UTF-8-kodierten JSON-Payloads.

- **Porttrennung:**
 - **Port 5005 (Roboterkanal):** Hochfrequente Gelenkwinkel, LED-Farben und Werkzeugbefehle.
 - **Port 5006 (Weltkanal):** Diskrete Ereignisse zur Objekt- und Szenenverwaltung (`spawn`, `delete`, `clear`).
- **Wurzelschema:** Jedes JSON-Paket enthält eine Versionsnummer (`"v": 1`) und eine Typkennung (`"type"`).
- **Zustandsbasierte Idempotenz:** Bewegungsnachrichten übertragen stets die absoluten

Zielwinkel aller vier Achsen. Ein einzelner Paketverlust führt somit nicht zu kumulativen Posenfehlern, da das Folgepaket den Roboter sofort auf die korrekte Position bringt.

9.3. Threading- und Nebenläufigkeitskonzept in Unity

Unity erlaubt Zugriffe auf Szenenobjekte und Physikkomponenten ausschliesslich aus dem Engine-Hauptthread. Netzwerkaufrufe dürfen den Rendering-Zyklus jedoch nicht blockieren.



- **Entkopplung:** Empfangene UDP-Pakete werden im Hintergrund-Thread deserialisiert und unter Verwendung eines Mutex (`lock_lock`) in eine thread-sichere Warteschlange eingereiht.
- **Abarbeitung via Coroutine:** Der `CommandRunner` holt Befehle im Hauptthread ab. Bei synchronen Bewegungen (`sync_move_joints`) pausiert die Coroutine mit `yield return`, bis alle Gelenke innerhalb der Toleranz (`epsDeg = 0.5°`) liegen, wodurch nachfolgende Queue-Befehle präzise synchronisiert werden.

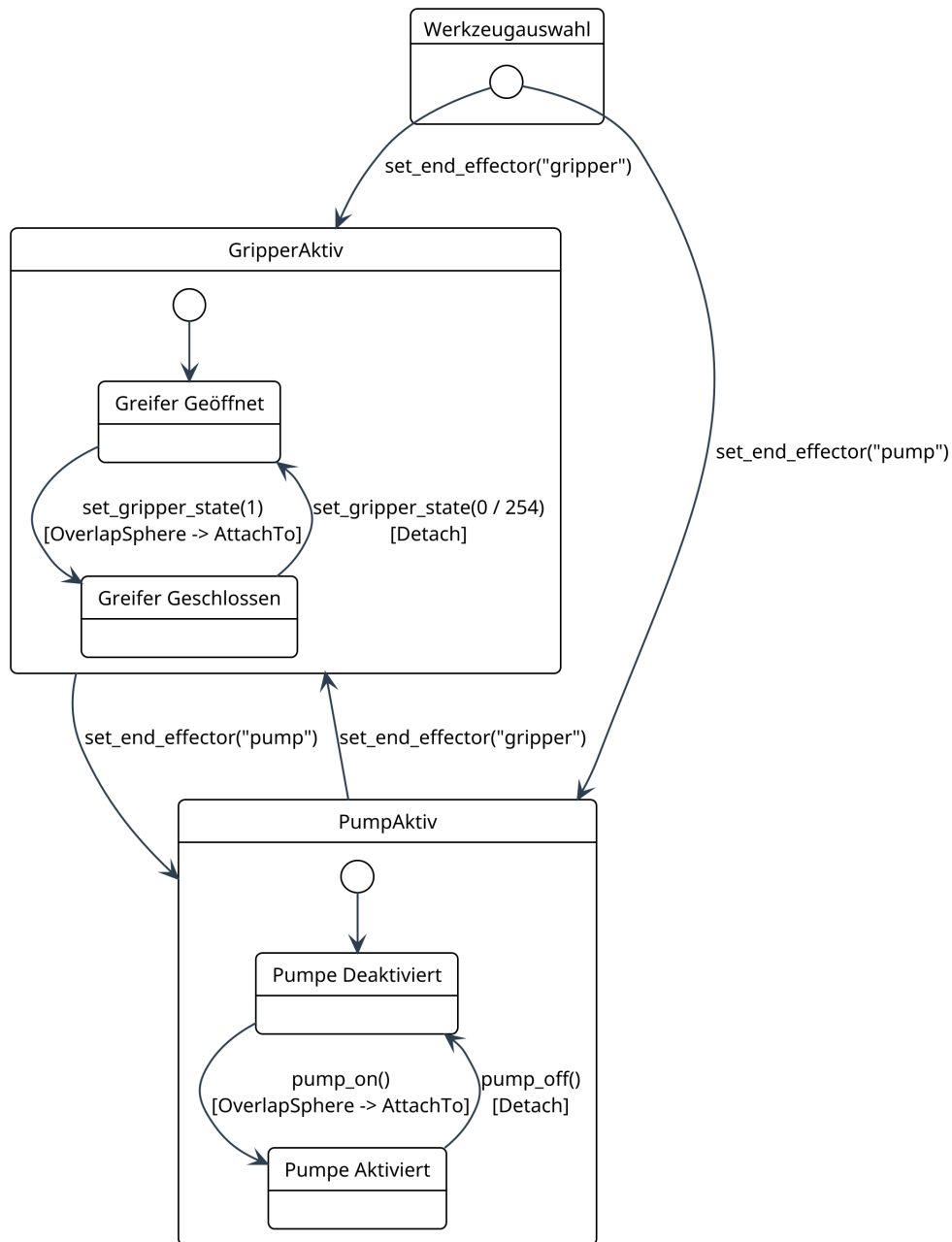
9.4. Zustands- und Telemetriedatenkonzept

Das System unterscheidet zwischen simuliertem Soll-Zustand und physikalischem Hardware-Ist-Zustand:

- **Modus virtual:** Der `MyPalletizerController` speichert gesetzte Winkel im lokalen Array `_sim_angles`. Die Methode `get_angles()` liefert diese Werte unmittelbar zurück. Dadurch arbeitet die API auch ohne Netzwerk-Rückkanal deterministisch.
- **Modus real:** `get_angles()` fragt die tatsächlichen Ist-Winkel über die serielle Schnittstelle von den Roboter-Servos ab.
- **Modus both:** `get_angles()` gibt einen formatierten String mit beiden Datensätzen (`Real` und `Sim`) zurück, was einen direkten Soll-Ist-Vergleich im Terminal erlaubt.

9.5. Endeffektor- und Greifkonzept

Die Abstraktion verschiedener Werkzeuge erfolgt über ein einheitliches Endeffektormodell:



- **Sichtbarkeitssteuerung:** Der `RobotToolManager` aktiviert das 3D-Modell des gewählten Werkzeugs (`gripperTool` bzw. `pumpTool`) und deaktiviert das andere.
- **Objekterfassung:** Beide Werkzeuge nutzen `Physics.OverlapSphere`, um interaktionsfähige Objekte mit der Komponente `GrabbableObject` im Arbeitsradius zu identifizieren.
- **Kinematische Kopplung:** Erfasste Objekte werden hierarchisch an den Werkzeugkopf gekoppelt (`obj.AttachTo(attachPoint)`), wobei deren Schwerkraft deaktiviert wird. Beim Lösen (`Detach`) wird die Schwerkraft reaktiviert.

9.6. Welt- und Szenarienkonzept

- **Dynamische Körper:** Über `Box`, `Sphere` und `Cylinder` können Objekte mit frei wählbarer

Position, Rotation, Skalierung und RGB-Farbe platziert werden.

- **Automatische Komponenten:** Der `WorldSpawnManager` stattet instanziierte Prefabs automatisch mit `Rigidbody`, Kollisionsgeometrie und `GrabbableObject` aus.
- **Reproduzierbare Presets:** Über `World.load_preset()` können standardisierte Übungsaufbauten (`three_blocks`, `pick_and_place_basic`, `sorting_task`) geladen werden.

9.7. Schutz- und Validierungskonzept

- **Fail-Fast Parameterprüfung:** Fehlende Ports oder fehlerhafte 3D-Vektoren lösen sofort eine `InvalidConfigError`-Exception aus.
- **Software-Clamping:** Gelenkwinkel werden vor dem Versand auf die exakten Hardwarespezifikationen begrenzt (J1: $-160..160^\circ$, J2: $0..90^\circ$, J3: $-92..60^\circ$, J4: $-360..360^\circ$). Geschwindigkeiten werden auf $1..100$ und RGB-Werte auf $0..255$ begrenzt.

9.8. Test- und Qualitätssicherungskonzept

- **Python Unit Tests:** Automatisierte Tests mit `pytest` validieren die Protokollerstellung (`test_protocol.py`), Roboter-API (`test_robot.py`) und Weltverwaltung (`test_world.py`) unter Verwendung von Sockets- und Hardware-Mocks.
- **Unity EditMode Tests:** NUnit-Tests (`SerializationTests`, `ComponentTests`) prüfen die Deserialisierung von JSON-Strings und das Vorhandensein erforderlicher Komponenten.
- **Systemtests:** Standardisierte End-to-End-Testszzenarien (z. B. Pick-and-Place-Zyklen) sind im GitHub-Wiki dokumentiert.

9.9. Usability und Developer Experience

- **Context-Manager:** Die Klassen `Robot` und `World` implementieren `enter` und `exit`, wodurch Sockets und Hardwareverbindungen bei Skriptende oder im Fehlerfall zuverlässig geschlossen werden (`with Robot(mode=RobotMode.VIRTUAL) as robot:`).
- **Typisierung:** Durchgängige Typannotationen und Enums (`RobotMode`, `EndEffector`) unterstützen die Codevervollständigung in gängigen IDEs.

10. Architecture Decisions

Die folgenden Architectural Decision Records (ADRs) dokumentieren die zentralen Richtungsentscheidungen, deren Begründung sowie die resultierenden Konsequenzen.

10.1. ADR-1: Einheitliche Python-API für Simulation und Hardware

- **Status:** Akzeptiert
- **Kontext:** Steuerungsskripte sollen ohne Codeänderungen zwischen Simulation und physischer Hardware übertragbar sein.

- **Entscheidung:** Die Klassen `Robot` und `World` stellen eine einheitliche Fassade bereit. Das Routing (`virtual`, `real`, `both`) erfolgt intern im `MyPalletizerController`.
- **Betrachtete Alternativen:**
 1. Getrennte Bibliotheken für Simulation und Hardware.
 2. Direkte Weitergabe der Herstellerbibliothek `pymycobot`.
- **Begründung:** Eine gemeinsame Schnittstelle minimiert den Lernaufwand und ermöglicht die direkte Wiederverwendung von Beispielcode.
- **Konsequenzen:**
- **Positiv:** Identische Syntax für alle Betriebsmodi; Entkopplung des Benutzercodes von Netzwerk- und Protokolldetails.
- **Negativ:** Zusätzliche Abstraktionsschicht zur Verwaltung von Moduslogik und Validierung.

10.2. ADR-2: Kapselung der Herstellerbibliothek `pymycobot`

- **Status:** Akzeptiert
- **Kontext:** Die herstellereigene Bibliothek `pymycobot` steuert den physischen Roboter, unterliegt jedoch herstellerseitigen API-Änderungen.
- **Entscheidung:** `pymycobot` wird ausschliesslich intern im `MyPalletizerController` gekapselt.
- **Betrachtete Alternativen:**
 1. Direkte Verwendung von `pymycobot` durch den Anwender.
 2. Komplette Neuentwicklung eines eigenen seriellen Treibers.
- **Begründung:** Die Kapselung schützt die öffentliche API vor herstellerspezifischen Schnittstellenbrüchen und ermöglicht zentrales Software-Clamping.
- **Konsequenzen:**
- **Positiv:** Stabile, typisierte Schnittstelle; zentrale Fehlerbehandlung.
- **Negativ:** Neue herstellerspezifische Funktionen müssen manuell in die Fassade integriert werden.

10.3. ADR-3: Hybride Betriebsmodi (`virtual`, `real`, `both`)

- **Status:** Akzeptiert
- **Kontext:** Es wird sowohl isoliertes Offline-Testing als auch der direkte Vergleich zwischen 3D-Modell und physischer Bewegung benötigt.
- **Entscheidung:** Unterstützung von drei festen Betriebsarten: `virtual` (nur Unity), `real` (nur Hardware) und `both` (Parallelbetrieb).
- **Betrachtete Alternativen:**
 1. Reine Simulationslösung.
 2. Umschalten über externe Konfigurationsdateien statt nativer Klassenparameter.

- **Begründung:** Der Modus **both** erlaubt den synchronen visuellen Abgleich von Kinematikmodell und Servo-Verhalten im Labor.
- **Konsequenzen:**
- **Positiv:** Hohe Flexibilität für Studierende im Selbststudium und im Labor.
- **Negativ:** Im Modus **both** können minimale zeitliche Abweichungen zwischen Simulation und Hardware entstehen.

10.4. ADR-4: Trennung der Kommunikationskanäle (UDP Ports 5005 und 5006)

- **Status:** Akzeptiert
- **Kontext:** Das System überträgt zeitkritische Gelenkbefehle sowie datenintensivere Szenen- und Objektbefehle.
- **Entscheidung:** Roboterkommandos werden über UDP-Port **5005** gesendet, Weltbefehle über UDP-Port **5006**.
- **Betrachtete Alternativen:**
 1. Gemeinsamer UDP-Kanal für alle Nachrichtentypen.
 2. Einsatz von TCP oder WebSockets.
- **Begründung:** Die Porttrennung verhindert, dass das Laden grosser Szenarien oder Presets die kontinuierliche Roboterkinematik blockiert.
- **Konsequenzen:**
- **Positiv:** Klare funktionale Entkopplung; keine Latenzspitzen bei der Roboterbewegung.
- **Negativ:** Zwei lokale Netzwerkports müssen freigehalten und konfiguriert werden.

10.5. ADR-5: Bereitstellung vordefinierter Szenarien über Presets

- **Status:** Akzeptiert
- **Kontext:** In Übungen müssen reproduzierbare Ausgangssituationen für Pick-and-Place-Aufgaben schnell geladen werden können.
- **Entscheidung:** Standardszenarien (**three_blocks**, **pick_and_place_basic**, **sorting_task**) werden als Python-Presets definiert und über **World.load_preset()** bereitgestellt.
- **Betrachtete Alternativen:**
 1. Manuelles Platzieren aller Objekte im Unity-Editor vor Programmstart.
 2. Laden ausschliesslich aus externen JSON-Dateien.
- **Begründung:** Codebasierte Presets ermöglichen deterministische und automatisierbare Übungsabläufe direkt aus dem Steuerungsskript.
- **Konsequenzen:**

- **Positiv:** Sofort einsatzbereite, reproduzierbare Laboraufgaben.
- **Negativ:** Presets müssen bei Erweiterung der 3D-Körper-Bibliothek nachgepflegt werden.

10.6. ADR-6: Einsatz von **ArticulationBody** für die Gelenkphysik in Unity

- **Status:** Akzeptiert
- **Kontext:** Der Roboterarm besteht aus vier rotatorisch verbundenen Starrkörpern, die Lasten aufnehmen und stabil bewegt werden müssen.
- **Entscheidung:** Verwendung von **ArticulationBody**-Komponenten statt herkömmlicher **Rigidbody** - und **HingeJoint**-Ketten.
- **Betrachtete Alternativen:**
 1. Rein visuelle Rotation über **Transform**-Hierarchien ohne Physikberechnung.
 2. Physiksimulation über Standard-**Rigidbody** mit Gelenken.
- **Begründung:** **ArticulationBody** wurde speziell für Robotiksimulationen entwickelt und verhindert numerische Gelenkdehnungen zuverlässig.
- **Konsequenzen:**
 - **Positiv:** Stabiles Mehrkörperverhalten; exakte Gelenkbeschränkungen.
 - **Negativ:** Höhere Konfigurationskomplexität im Unity-Inspector.

10.7. ADR-7: Pragmatische Nutzung herstellerseitiger CAD-Modelle mit Kompensation

- **Status:** Akzeptiert
- **Kontext:** Die bereitgestellten 3D-CAD-Modelle des Herstellers weisen minimale geometrische Toleranzen zur Hardware auf.
- **Entscheidung:** Verwendung der Hersteller-CAD-Modelle mit softwareseitiger Ausrichtungskompensation (**GripperDownConstraint**) in Unity.
- **Betrachtete Alternativen:**
 1. Vollständige Neukonstruktion aller 3D-Modelle im CAD-System.
 2. Verwendung stark vereinfachter Platzhalterkörper.
- **Begründung:** Eine Neukonstruktion stand ausserhalb des Projektzeitrahmens. Die softwareseitige Ausrichtung liefert eine für Ausbildungszwecke vollkommen ausreichende visuelle und funktionale Genauigkeit.
- **Konsequenzen:**
 - **Positiv:** Realistischer optischer Eindruck bei minimalem Modellierungsaufwand.
 - **Negativ:** Für mikrometergenaue Kollisionsprüfungen ist das Modell nicht geeignet.

10.8. ADR-8: Verzicht auf optische Kameranimulation zugunsten der Kernkinematik

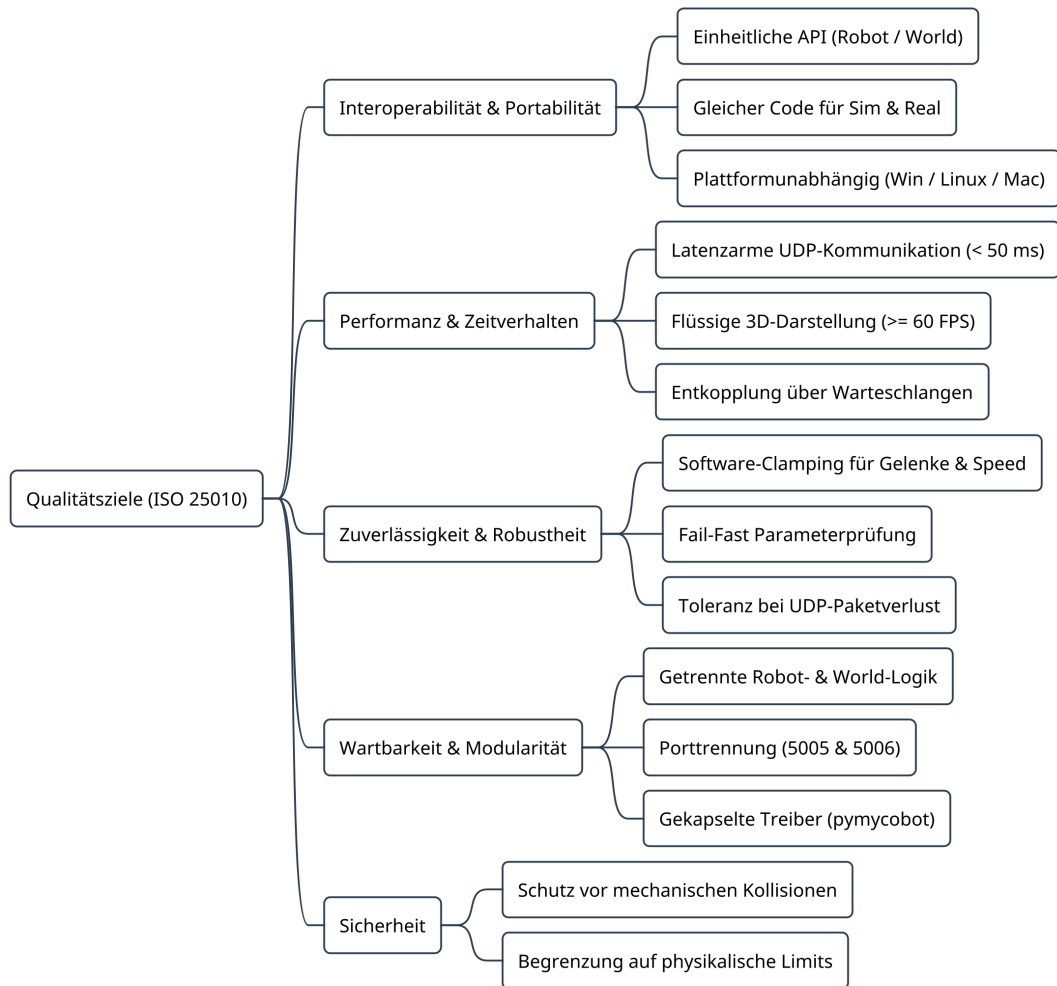
- **Status:** Akzeptiert
- **Kontext:** Der reale MyPalletizer kann mit einer USB-Kamera betrieben werden. Kameranimulationen unterliegen jedoch komplexen Umweltfaktoren (Beleuchtung, Rauschen).
- **Entscheidung:** Das System fokussiert auf Kinematik, Werkzeuge und Objektmanipulation; auf eine Kameranimulation in Unity wird verzichtet.
- **Betrachtete Alternativen:**
 1. Virtuelle Unity-Kamera mit Bildstream an OpenCV.
- **Begründung:** Das Hauptziel ist die Programmierung von Manipulations- und Sortieraufgaben. Bildverarbeitung wird an der HSLU direkt an realer Hardware unterrichtet.
- **Konsequenzen:**
- **Positiv:** Schlanke Architektur, geringe Renderlast, fokussierte Codebasis.
- **Negativ:** Bildverarbeitungs-Pipelines können nicht virtuell vortrainiert werden.

11. Quality Requirements

Die Qualitätsanforderungen konkretisieren die Qualitätsziele aus Kapitel 1 anhand eines Qualitätsbaums sowie messbarer Qualitätsszenarien.

11.1. Quality Tree

Der Qualitätsbaum ordnet die Anforderungen den ISO/IEC 25010 Qualitätsmerkmalen zu:



11.2. Quality Scenarios

ID	Merkmal	Szenario (Stimulus & Kontext)	Messbares Akzeptanzkriterium
QS-1	Performanz	Ein Benutzer sendet Gelenkbewegungen (send_angles) über die Python-API an die laufende Unity-Simulation.	Die visuelle Bewegung des Roboterarms beginnt innerhalb von weniger als 50 ms .
QS-2	Portabilität	Ein Steuerungsskript wird unter Windows entwickelt und anschliessend auf macOS oder Linux ausgeführt.	Das Skript läuft ohne Codeanpassungen fehlerfrei aus.
QS-3	Robustheit	Die Unity-Simulation wird während der Programmausführung unerwartet beendet oder ist noch nicht gestartet.	Das Python-Skript blockiert nicht und bricht nicht unkontrolliert ab.
QS-4	Eingabeschutz	Ein Benutzer übergibt einen Gelenkwinkel von 200° für Joint 1 (zulässig: -160°..160°).	Der Controller fängt den Wert ab und klemmt ihn automatisch auf 160° (Software-Clamping).

ID	Merkmal	Szenario (Stimulus & Kontext)	Messbares Akzeptanzkriterium
QS-5	Benutzbarkeit	Ein neuer Anwender ohne Unity-Vorkenntnisse installiert das Paket und führt ein Beispielskript aus.	Die erste Roboterbewegung in der Simulation ist in unter 15 Minuten erfolgreich ausgeführt.
QS-6	Wartbarkeit	Ein Entwickler ergänzt einen neuen administrativen Weltbefehl in Python und C#.	Die Änderung betrifft ausschliesslich das protocol -Modul sowie den WorldSpawnManager , ohne bestehende Roboterklassen zu modifizieren.
QS-7	Funktionale Korrektheit	Ein Pick-and-Place-Ablauf wird synchron im Modus both ausgeführt.	Gelenkrichtungen und Werkzeugaktivierungen in der Simulation stimmen visuell plausibel mit der Hardware überein.
QS-8	Fehlertoleranz	Während einer Bewegung geht ein einzelnes UDP-Datagramm im lokalen Netzwerk verloren.	Das Folgepaket führt den Roboterarm nahtlos auf die korrekte Zielbahn (kein Posenversatz).
QS-9	Sicherheit	Ein Benutzer übergibt Geschwindigkeitswerte von 250 oder negative RGB-Werte.	Geschwindigkeiten werden auf 1..100 und RGB-Werte auf 0..255 geklemmt.
QS-10	Reproduzierbarkeit	Ein Praktikumsszenario soll deterministisch zurückgesetzt werden.	Der Aufruf World.load_preset("three_blocks") reinigt die Szene und instanziiert die Objekte an exakt definierten Startkoordinaten.

11.3. Architektonische Zielkonflikte (Trade-offs)

- **Latenz vs. Garantierte Zustellung:** Verbindungsloses UDP priorisiert minimale Latenz gegenüber garantierter TCP-Paketzustellung; die Zustands-Idempotenz kompensiert diesen Verlust.
- **Einfachheit der API vs. Hardware-Transparenz:** Die High-Level-Fassade verbirgt feingranulare herstellerspezifische Sonderfunktionen von **pymycobot**, um eine einheitliche und intuitive Schnittstelle zu bieten.
- **Visuelle Plausibilität vs. Hochpräzise Dynamik:** Die Simulation priorisiert eine performante Echtzeitdarstellung in Unity gegenüber einer rechenintensiven Finite-Elemente-Simulation.

12. Risks and Technical Debts

Dieses Kapitel dokumentiert bekannte architektonische und technologische Risiken, technische Schulden sowie Massnahmen zu deren Beherrschung.

12.1. Architektur-Risiken

Risiko	Ursache und Auswirkung	Massnahme zur Risikominderung
Paketverlust bei UDP-Kommunikation	UDP garantiert weder Zustellung noch Reihenfolge.	Jedes Datagramm enthält den vollständigen absoluten Zustand (j1 bis j4). Das nächste empfangene Paket korrigiert die Pose ohne bleibenden Versatz.
Open-Loop-Steuerung der Simulation	Die Python-API sendet Befehle an Unity, ohne eine physikalische Quittung abzuwarten.	Für den Lehrkontext ist das ausreichend. Der lokale Zustandsspeicher _sim_angles stellt sicher, dass get_angles() unmittelbar konsistente Werte liefert.
Laufzeit-Asynchronität im Modus both	Unity-Physik und reale Servomotoren haben leicht unterschiedliche Trägheitsmomente und Ansprechzeiten.	Beide Zielsysteme erhalten identische Zielwinkel und Geschwindigkeitsstufen; für synchrone Bewegungen blockiert sync_move_joints auf Python-Seite bis zur Fertigstellung.
Port-Kollisionen	Standard-UDP-Ports 5005 und 5006 könnten belegt sein.	Die Ports sind im Python-Konstruktor (RobotConfig , World) und in den Unity-Inspektoren frei konfigurierbar.

12.2. Technologische und Implementierungsrisiken

Risiko	Ursache und Auswirkung	Massnahme zur Risikominderung
CAD-Modell-Toleranzen	Die 3D-Modelle des Herstellers weisen minimale Abweichungen zur physischen Hardware auf.	Softwareseitige Ausrichtungskompensation durch GripperDownConstraint in Unity; transparente Dokumentation des Kinematikfokus.
Fehlende Kamerasimulation	Die optionale Hardware-Kamera wird in Unity nicht simuliert.	Klare funktionale Abgrenzung: Das Projekt fokussiert auf Manipulations- und Bewegungslogik. Bildverarbeitung wird separat an der realen Hardware unterrichtet.

Risiko	Ursache und Auswirkung	Massnahme zur Risikominderung
Abhängigkeit von <code>pymycobot</code>	Fehler in der Herstellerbibliothek könnten die Hardwareansteuerung beeinträchtigen.	Kapselung von <code>pymycobot</code> im <code>MyPalletizerController</code> . Bei Bedarf kann die Implementierung gegen eine eigene serielle Treiberschicht ausgetauscht werden.
Thread-Sicherheit in Unity	Fehlerhafte Zugriffe aus dem Netzwerk-Thread auf Unity-Objekte führen zu Engine-Crashes.	Strikte Trennung über threadsichere Sperren (<code>lock</code>) und Queues (<code>_commandQueue</code>), die ausschliesslich im Unity-Hauptthread (<code>CommandRunner</code>) abgearbeitet werden.

12.3. Technische Schulden und Weiterentwicklung

- **Automatisierung von End-to-End-Systemtests:** Unit-Tests in Python und EditMode-Tests in Unity sind automatisiert. Systemtests (Zusammenspiel über UDP) werden derzeit manuell nach Wiki-Testprotokoll durchgeführt. Langfristig ist ein Headless-Unity-Testrunner in GitHub Actions geplant.
- **Vereinfachtes Greifermodell:** Werkzeuge nutzen ein kinematisches Kopplungsmodell (`AttachTo`), anstatt komplexe Reibungskräfte physikalisch zu berechnen. Für Lehrzwecke ist dies stabil und performant; ein Reibungsmodell kann bei Bedarf modular nachgerüstet werden.
- **Bidirektionaler Telemetrie-Rückkanal:** Unity sendet derzeit keine Messwerte an Python zurück; `get_angles()` basiert in der Simulation auf `_sim_angles`. Eine spätere Erweiterung um einen Rückkanal (z. B. auf Port `5007`) ist architektonisch vorbereitet.

13. Glossary

Die folgende Tabelle erläutert die im Dokument verwendeten Fachbegriffe, Modulnamen und Akronyme in alphabetischer Reihenfolge:

Begriff / Akronym	Definition und Projektkontext
AI-ROBOTICS	Folgemodul an der Hochschule Luzern, in welchem der digitale Zwilling und die Python-API für praktische Robotik- und KI-Aufgaben eingesetzt werden.
APPLAI	Applied Artificial Intelligence. Ausbildungsmodul an der Hochschule Luzern, in dessen Rahmen dieser digitale Zwilling entwickelt wurde.
ArticulationBody	Spezialisierte Unity-Physikkomponente zur Modellierung verbundener Starrkörperketten (Gelenke) mittels Featherstone-Algorithmus.
Clamping (Software-Begrenzung)	Automatisches Begrenzen numerischer Parameter (Gelenkwinkel, Geschwindigkeiten, RGB-Werte) auf physikalisch und mechanisch sichere Grenzen.
Coroutine	C#-Sprachmittel in Unity zur Ausführung von asynchronen Abläufen über mehrere Frames hinweg (<code>yield return</code>) ohne Thread-Blockaden.

Begriff / Akronym	Definition und Projektkontext
Digitaler Zwilling (Digital Twin)	Softwarebasiertes, virtuelles 3D-Abbild des MyPalletizer 260 M5 zur Echtzeitsimulation von Kinematik, Werkzeugen und Umgebungsinteraktionen.
Endeffektor	Am Roboterarm montiertes Werkzeug: Parallelgreifer (Gripper) oder Vakuumpumpe (Suction Pump).
Fail-Fast	Entwurfsprinzip, bei dem fehlerhafte Parameter sofort beim Methodenaufruf durch eine Exception abgefangen werden.
GrabbableObject	Unity-Komponente zur Kennzeichnung von Objekten, die von Greifer oder Vakuumpumpe aufgenommen werden können.
GripperDownConstraint	Unity-Komponente zur softwareseitigen Ausrichtungskompensation, damit der Werkzeugkopf bei jeder Armstellung vertikal bleibt (Palettiermechanik).
HSLU	Hochschule Luzern (Departement Informatik).
JSON	JavaScript Object Notation . Textbasiertes Datenformat zur Strukturierung der UDP-Nachrichten zwischen Python und Unity.
Kinematische Kette (4-DOF)	Reihe starr verbundener Roboterglieder mit vier rotatorischen Freiheitsgraden (Degrees of Freedom).
MyPalletizer 260 M5	4-Achs-Palettierroboterarm der Firma Elephant Robotics , basierend auf dem M5Stack-Mikrocontroller (ESP32).
Open-Loop-Steuerung	Steuerungsansatz ohne kontinuierlichen Rückkanal; Kommandos werden ohne explizite Empfangsquittung abgesetzt.
Preset	Vordefiniertes, reproduzierbares Simulationsszenario (z. B. three_blocks), das programmgesteuert in die Unity-Szene geladen werden kann.
pymycobot	Offizielle Python-Treiberbibliothek von Elephant Robotics zur seriellen Steuerung des MyPalletizer 260 M5.
UDP	User Datagram Protocol . Verbindungsloses, latenzarmes Netzwerkprotokoll zur Übertragung von Steuerpaketen.
Unity	3D-Echtzeit- und Physik-Engine zur Visualisierung und Simulation des Roboters.